RPGAgent: Driving Coherent Story-to-Play Generation with an LLM-Based Multi-Agent System

Shunan Zhang
School of Design
Hunan University
Changsha, Hunan, China
zhangshunan@hnu.edu.cn

Yi Xiao*
School of Design
Hunan University
Changsha, Hunan, China
yixiao_csee@hnu.edu.cn

Ruoxuan Ma
School of Design
Hunan University
Changsha, Hunan, China
roxanne@hnu.edu.cn

Chi-sing Leung
Electronic Engineering
City University of Hong Kong
Hong Kong, Hong Kong
eeleungc@cityu.edu.hk

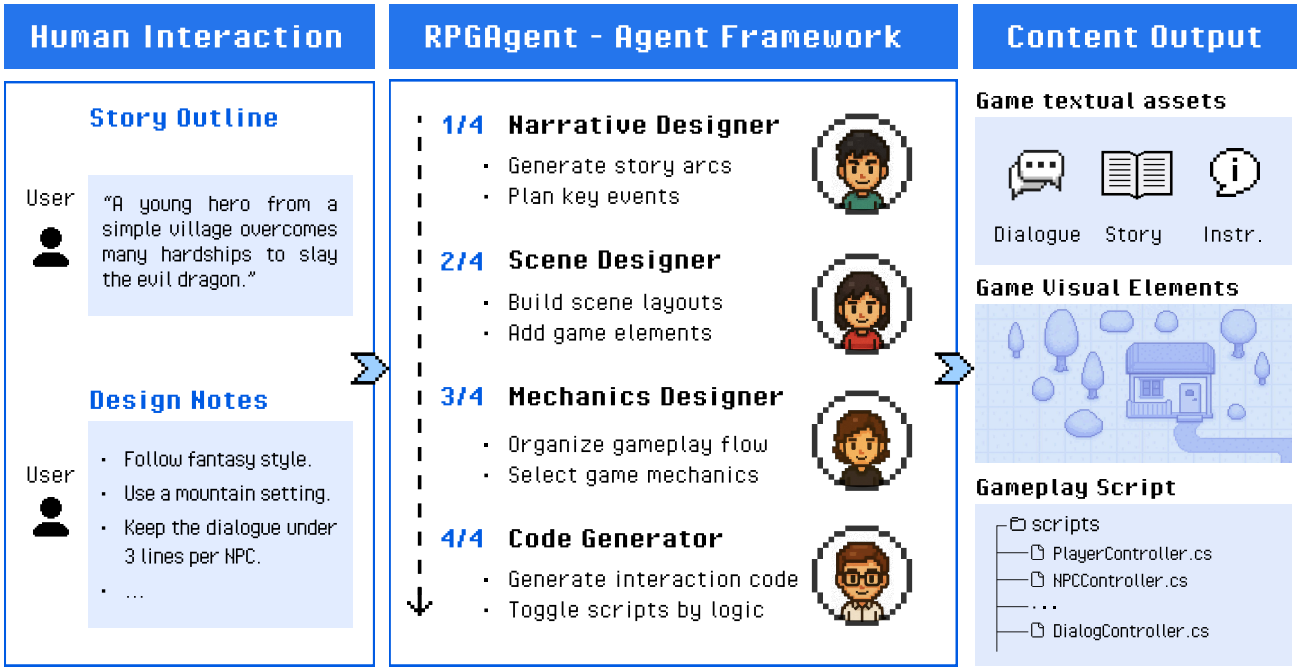


Figure 1: In this work, we present *RPGAgent*, an LLM-driven multi-agent tool that turns designer prompts into coherent game content. Designers provide story outlines and notes; specialized agents craft narrative, scenes, and mechanics; the system outputs integrated text assets, visual elements, and gameplay scripts.

Abstract

Recent advances in LLMs have enabled new possibilities for creative content generation, yet their use in game design is often limited by poor integration across creative components, particularly for novice designers aiming to rapidly prototype playable concepts. Guided by the Elemental Tetrad framework, we present RPGAgent, an LLM-driven multi-agent system specifically designed to assist novice game creators in transforming a short story outline into a

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.

CHI '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2278-3/26/04
https://doi.org/10.1145/3772318.3790326

playable game. Specialized agents exchange structured data to generate coherent narrative, scene, and gameplay mechanics, ensuring structural correctness and consistency between story and world. By combining LLM-based generation with procedural content creation, the system offers a controllable and interpretable workflow. In a within-subjects study with 18 participants, RPGAgent outperformed a GPT-assisted baseline in both user experience and creative satisfaction during the prototyping of playable RPGs. These results demonstrate the potential of collaborative multi-agent frameworks for structured, AI-assisted game design.

CCS Concepts

• **Human-centered computing → User interface toolkits**.

Keywords

Multi-Agent Systems, AI-assisted Game Design, Procedural Content Generation, Creativity Support Tools

ACM Reference Format:

Shunan Zhang, Yi Xiao, Ruoxuan Ma, and Chi-sing Leung. 2026. RPGAgent: Driving Coherent Story-to-Play Generation with an LLM-Based Multi-Agent System. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26), April 13–17, 2026, Barcelona, Spain.* ACM, New York, NY, USA, 22 pages. https://doi.org/10.1145/3772318.3790326

1 Introduction

Recent advances in AI-generated content (AIGC) are transforming creative workflows across numerous domains, prompting both amateur and professional creators to widely adopt assistive AI tools for generating text, code, images, and 3D assets. At the forefront of this evolution, Large Language Models (LLMs) have empowered a new generation of content creation tools, enabling flexible and context-aware generation across diverse modalities [1, 32]. These models have been increasingly integrated into creative processes, allowing for the rapid prototyping of narratives, dialogues, and interactive elements through natural language prompts, thereby lowering barriers to entry and accelerating ideation [50, 54]. However, while LLMs excel at producing semantically rich content, their monolithic structure struggles to process heterogeneous content modalities within a single context window. This susceptibility to "context drift" disrupts cross-domain coherence, often resulting in structural inconsistencies and fragmented outputs [27, 74, 83].

To address these complexities, a particularly promising paradigm has emerged: the organization of LLMs into multi-agent systems, where specialized agents collaborate synergistically to tackle multifaceted tasks [23, 37]. This paradigm has successfully automated workflows in collaborative writing [37], software development [24, 29], and video production [81]. By decomposing monolithic processes, these systems facilitate targeted generation and scalability [21]. However, in the domain of game development, existing multi-agent applications often focus on simulating human production processes (e.g., role-playing a studio) [10]. Few approaches prioritize the holistic integration of diverse creative elements within the game artifact itself, or function explicitly as creativity support tools embedded in the development workflow.

According to Jesse Schell's Elemental Tetrad, a compelling game experience emerges from the balanced integration of four fundamental components: *mechanics*, *story*, *aesthetics*, and *technology* [57]. Prior work similarly emphasizes that successful games require the seamless alignment of narrative, spatial design, and gameplay mechanics [6, 40]. In this paper, we operationally define "coherence" as the generation of visual layouts (Aesthetics) and gameplay scripts (Mechanics) that are logically consistent with, and directly derived from, the core narrative's elements (such as locations, characters, and items). However, current generative AI approaches often struggle to achieve this cohesion. Monolithic LLMs frequently lack the architectural capacity to unify these diverse creative components. While combining traditional Procedural Content Generation (PCG) techniques with LLMs can improve thematic richness and variety, such integration is often constrained to scene-level generation and lacks a top-down framework to ensure that all content is holistically guided by the overarching narrative [43, 68]. For less-experienced or novice game developers, this limitation is even more pronounced: without a unified framework, they must invest substantial effort in learning complex tools and manually resolving inconsistencies, which slows rapid prototyping and constrains creative exploration. One potential solution is to replace the single monolithic model with a decomposed framework in which specialized agents collaborate on distinct generative tasks within a coherent, narrative-driven workflow.

Drawing inspiration from this multi-agent paradigm, we have developed *RPGAgent*[1], a framework that combines collaborating LLM-based agents with PCG techniques to address the specific demands of narrative-driven game design. Taking 2D RPG-style games as the representative genre, the framework accepts a brief game story outline and design notes, and decomposes the creative process by assigning tasks such as narrative structuring, spatial layout synthesis, gameplay logic design, and code generation to dedicated agents. Designed to support less-experienced developers, RPGAgent transforms a designer's high-level vision into a coherent, playable prototype while enabling rapid prototyping and human oversight. To investigate the dynamics and potential of this approach as a novel method for game content generation, we conducted a study guided by the following research questions (RQs):

- **RQ1:** In what ways does RPGAgent's integrated 'story-to-play' pipeline support or hinder designers' perceived usability and prototyping efficiency?
- **RQ2:** How do designers experience and describe the creative process when using RPGAgent, particularly regarding idea generation, exploration, and maintaining creative control?
- **RQ3:** How do designers perceive, interact with, and control the RPGAgent system, and what key tensions emerge between automation and designer agency within this co-creative workflow?

Our findings indicate that RPGAgent's multi-agent framework successfully enhances the game design process by integrating disparate creative elements into a coherent whole. The structured delegation of tasks to specialized agents creates a clear link between narrative, scene, and mechanics, while the modular pipeline

[1]We plan to release the necessary source code, prompts, and assets after publication.

provides designers with intuitive control. This fosters a fluid co-creative workflow where designers use both high-level prompts to set a vision and procedural controls to fine-tune outputs, maintaining a strong sense of creative agency. Our evaluation confirms that this approach significantly boosts design efficiency and creative exploration. However, while participants praised the system as a co-creative partner, our results also uncovered a key tension between the desire for full automation and the need for granular designer oversight-a critical challenge for future work.

Based on these findings, the main contributions of this work are:

- We present *RPGAgent*, a novel framework that embeds LLM-based multi-agent collaboration within a game engine, accelerating early-stage RPG prototyping and fostering creative ideation.
- Building on the Elemental Tetrad framework, we propose a structured strategy for multi-agent content generation that ensures consistency across Story, Aesthetics, Mechanics, and Technology in the game.
- By combining PCG techniques with coordinated multi-agent workflows, the system enhances spatial variety while maintaining structural consistency in game prototypes.
- Our evaluation demonstrates that the RPGAgent framework effectively bridges the gap between abstract narratives and playable prototypes, making the early design process faster, more coherent, and more inspiring for creators.

2 Background and Related Work

2.1 Background

Digital games rely on a set of interdependent elements that must work in concert to form a cohesive player experience. Building on frameworks such as Jesse Schell's Elemental Tetrad [57], prior research highlights how rules and systems, narrative progression, sensory presentation, and enabling technologies shape one another in practice [5, 53, 65, 75]. A change in a game's mechanics can alter its narrative pacing, while aesthetic choices directly influence the player's emotional connection to the story. The central challenge in game creation, therefore, is maintaining this holistic coherence, especially during early-stage prototyping where designers translate high-level concepts into interactive scenes and gameplay mechanics [33, 76]. A misalignment between the narrative that is *told*, the scene that is *shown*, and the mechanics that are *played* can undermine the player's immersion and the intended emotional impact [25, 72, 75].

This challenge of alignment is especially pronounced in RPGs, where story, character development, aesthetics, and player agency are deeply interwoven [41, 84]. In titles such as *Chrono Trigger* or *Undertale*, achieving coherence among quests, environments, and mechanics is critical for narrative believability and immersion. Aesthetics, in particular, reinforce this consistency through mood, sound, and environmental storytelling; a tragic storyline, for example, is amplified by desaturated palettes and melancholic music. Poor alignment, such as dramatic events unfolding in overly whimsical or brightly cartoonish settings, can disrupt this illusion. RPGs therefore demand strong semantic consistency across all design dimensions to create a believable world.

2.2 Applications of LLMs in Game Development

LLMs are rapidly transforming interactive digital media, with their application in game development workflows being a particularly active area of research [15, 20, 82]. Initially leveraged for tasks such as dialogue generation and code autocompletion [28, 30], LLMs have recently begun shaping broader dimensions of game design, including content prototyping [39], environment creation [35, 64], and player behavior simulation [38, 49]. This transformation reflects a larger paradigm shift where artificial intelligence is no longer confined to backend tools but emerges as a co-creative partner across the production pipeline.

LLMs have found their most prominent application in the game domain within storytelling and narrative design. Mishra et al. [46] introduced WhatIF, a system that enables designers to co-author emergent, branching storylines with the aid of LLMs and visual editing tools. Marsden et al. [45] focused on the use of LLMs in simulating nuanced human dialogue, applying the method to serious games addressing social bias and discrimination. Wang et al. [77] developed ClueCart, a prompt engineering tool for guiding LLMs to generate narrative clues and quest texts, enabling game designers to iteratively control story logic and semantic consistency during content authoring. Complementing these approaches, Peng et al. [50] examined player-driven narrative emergence in LLM-powered game environments, emphasizing how dynamic storylines can be co-shaped through interactions between players and generative agents.

Beyond narrative, recent work has explored the generation of visual assets and gameplay mechanics using LLM-based systems. For instance, CrawLLM [85] presented a novel application in game re-theming, a method that uses LLMs to adapt game assets to new narratives and generate corresponding scenes. Earle et al. [14] introduced DreamGarden, an LLM-based planning system for Unreal Engine that translates high-level user prompts into hierarchical action plans which evolve dynamically through user feedback. Focusing on gameplay mechanics, Jennings et al. [31] demonstrated that runtime behavior generation can be embedded directly into game mechanics to enable emergent interaction.

To achieve the coherent integration of in-game elements, recent research has explored unified generation paradigms. World modeling represents a significant step toward this goal, using LLMs to simulate environments and agents for dynamic play [34]. However, these models often operate as "black boxes," making it difficult to export structured, editable assets for standard game engines. Similarly, linear approaches have been explored. For instance, Monolithic Single-Stream Generation attempts to produce entire game descriptions directly [22], while Conversational Co-Creation relies on unstructured chain-of-thought interactions to evolve designs step-by-step [3]. Yet, as the complexity of game design documents grows, monolithic agents struggle with "context window saturation" [15]. Furthermore, purely conversational methods rely on unstructured natural language handoffs, which lack rigorous intermediate validation mechanisms. This limitation makes it difficult to enforce strict structural consistency, often leading to error propagation and hallucinations [82]. Consequently, few frameworks successfully jointly model story progression, rule evolution, and

spatial structure, underscoring the need for modular, multi-agent collaboration.

2.3 Procedural Content Generation in Games

PCG refers to the algorithmic creation of game content with limited or indirect user input, offering an efficient alternative for building complex game worlds without overburdening designers. It plays a central role in modern game development, enabling the generation of levels, environments, quests, and challenges [19, 59, 69]. Traditional PCG methods often rely on predefined rules and randomness to produce varied and unpredictable content, particularly in genres like roguelikes [12, 62]. In contrast, search-based approaches use optimization algorithms to explore the solution space and generate content that meets specific design goals [70]. These have been applied to the procedural generation of missions [51], maps [2], and enemies [52]. Notably, *Game Forge* [18] addressed story-driven world generation by employing search-based optimization rather than LLMs to find and render a game world tailored to specific narrative structures.

Search-based PCG's reliance on handcrafted evaluation functions limits its generalizability. To address this, recent approaches have explored Procedural Content Generation via Machine Learning (PCGML) [7, 44, 68]. These models infer design principles by mimicking learned distributions from existing data, mainly including n-grams/Markov Models [13, 63], long short-term memory (LSTMs) [67], Transformers [47], Autoencoders [56], and Generative Adversarial Networks (GANs) [61, 71, 73]. PCGML also supports non-spatial genres, such as interactive fiction [16] and card game content. For instance, *Mystical Tutor* allows users to input partial card specifications and receive a complete card design; this is achieved using an encoder-decoder model as a denoising autoencoder to learn the conversion from partial to full specifications [66].

To further enhance high-level semantic understanding, recent research has begun integrating LLMs with PCG, aiming to inject semantic constraints and design intent into the generation process [43]. Much of this work focuses on translating natural language descriptions into 3D environments. This is achieved by using LLMs to extract spatial constraints for layout optimization [79], translate text into executable PCG blueprints with spline-based controls [64], or parse text into procedural actions for large-scale asset placement [86]. Other approaches have used LLMs to link narrative design directly to tile-based spatial layouts [48] or to simultaneously generate both game rules and levels [22]. These works primarily leverage LLMs to translate high-level user inputs into the structured representations required by PCG. While these works showcase the versatility of LLMs across specific domains, especially level generation, they often focus on isolated aspects. The challenge of integrating narrative, spatial, and gameplay mechanics into a unified, controllable generation pipeline remains largely underexplored. Therefore, our work aims to address this gap by proposing a framework that combines LLM-based multi-agent collaboration with PCG techniques to generate coherent, playable game prototypes from high-level story outlines.

2.4 Creativity Multi-Agent Support Tools

Although our work is situated in the domain of game design, it is inspired by broader advances in multi-agent systems for creative support and development assistance. In particular, we examine how recent LLM-based agent frameworks have been used across domains to coordinate content generation tasks. These systems consist of multiple specialized agents, such as those for ideation, critique, refinement, or multimodal synthesis, that collaborate to enhance semantic coherence and creative control [21]. Rather than functioning as monolithic generators, they emulate human collaborative practices by distributing responsibilities and enabling iterative feedback.

In the domain of writing and textual content creation [26, 37, 58], multi-agent systems have been applied to support both narrative composition and argumentative structuring. For example, Li et al. [37] demonstrated how different agents could assume rhetorical roles, such as providing claims or counterarguments, to collaboratively construct persuasive essays. In addition, recent research has actively explored multi-agent collaborative frameworks for visual and multimodal content generation [23, 80, 81]. For example, Xu et al. [81] proposed FilmAgent, which automates the virtual filmmaking pipeline by simulating director, actor, and cinematographer roles, enabling end-to-end production from scriptwriting to 3D scene rendering.

In the area of computational logic and code generation [24, 29, 55], multi-agent models have supported task decomposition and modular code development. For instance, systems like AgentCoder [24] assign specific roles (e.g., programmer, test designer), while ChatDev [55] simulates an entire software development team. More recently, this approach has been applied directly to game development [10], where frameworks like GameGPT also simulate a team of roles to automate development tasks.

Across these application domains, researchers have identified a common pattern in agent collaboration: role-based delegation, where agents specialize in specific tasks. This body of work suggests that such architectures can enhance creative support, interpretability, and user steerability. Drawing inspiration from this established approach, RPGAgent organizes the "story-to-play" pipeline by delegating responsibilities to modular agents (narrative, spatial, and mechanical), aiming to translate these advanced collaboration patterns into a practical, efficient tool for game creators.

3 RPGAgent: Design and Implementation

To operationalize our vision of collaborative game creation, we developed RPGAgent, a multi-agent system that integrates LLMs with PCG techniques. This section details the system's core design goals, its architecture, and the design and specific implementation of its workflow.

3.1 Design Goals

RPGAgent is founded on two core design goals:

The primary objective is to achieve Semantic Alignment and Designer Control. This goal ensures the cohesive integration of narrative arcs, procedural scene layouts, and gameplay mechanics. To

accomplish this, we employ a multi-agent framework where specialized agents-for narrative, scene, mechanics, and code generation-collaborate through structured interfaces. This design ensures that narrative progression drives the creation of relevant scenes and mechanics. Crucially, the architecture is designed to support mixed-initiative intervention, allowing designers to review, edit, and refine agent outputs at any stage. This grants designers a high level of control, ensuring the final product aligns with their creative vision.

The second goal is to Enable Creative Prototyping. The system is intended not merely to automate tasks for efficiency, but to act as a "co-creative partner" for designers. RPGAgent aims to accelerate the early stages of game design by generating a coherent foundation of narrative, scenes, and gameplay, thereby serving to inspire creative ideation. Designers can treat the agent-generated content as "building blocks for new ideas," allowing them to rapidly explore different concepts, iterate on their designs, and ultimately enhance their creative flexibility and satisfaction.

To realize these goals, RPGAgent is grounded in a clear theoretical framework that guides both its multi-agent architecture and interaction design.

3.2 Theoretical Foundation: The Elemental Tetrad

To address the challenge of coherence, our system's design is conceptually grounded in the Elemental Tetrad. We selected this framework because its synergistic balance of four fundamental elements provides a holistic model for the very components our system must integrate. These elements are the Mechanics (the rules and systems that define gameplay), the Story (the narrative arc that unfolds), the Aesthetics (the sensory experience and overall mood), and the Technology (the underlying implementation that makes the game possible). Standard generative AI approaches, often monolithic in nature, struggle to manage the complex interdependencies between these elements, frequently resulting in a "coherence gap." RPGAgent is designed to bridge this gap by transforming the Tetrad from a descriptive model into a prescriptive architecture for co-creation.

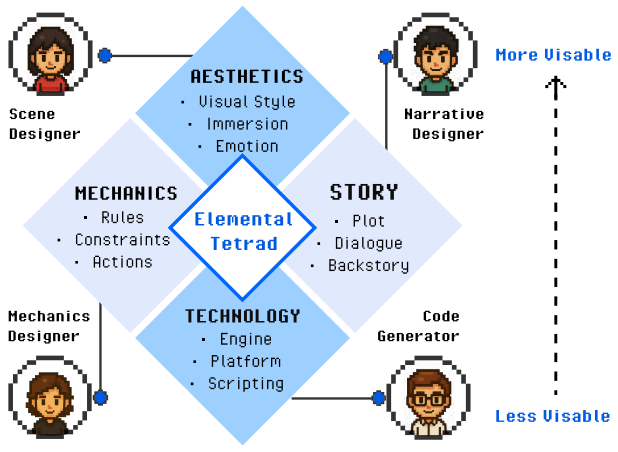


Figure 2: RPGAgent's multi-agent framework mapped to the Elemental Tetrad, organized by agent visibility.

RPGAgent operationalizes the Elemental Tetrad by dedicating a specialized agent to each core element, transforming the framework from a descriptive model into a prescriptive architecture for co-creation, as shown in Figure 2. The Narrative Designer is responsible for the Story element, the Scene Designer manages Aesthetics, the Mechanics Designer defines the game's Mechanics, and the Code Generator handles the underlying Technology. This direct mapping is central to our interaction design, which features varying levels of "agent visibility." Agents tied to tangible, creative work, like the Scene Designer (Aesthetics), are intentionally made "More Visible" through intuitive graphical interfaces that allow designers to sculpt the game world and receive immediate visual feedback. Conversely, agents handling technical complexity, like the Code Generator (Technology), are "Less Visible," operating as an automated backend process that translates high-level mechanics into code without requiring direct intervention.

This structure provides a crucial advantage: it facilitates principled negotiation between the elements of game design. Instead of a single model attempting to generate everything at once, the agents collaborate through structured interfaces, ensuring, for example, that a narrative beat designed by the Story agent directly informs the level layout from the Scene agent and the interaction rules from the Mechanics agent. This deliberate, theory-driven process ensures not just alignment, but a deeper, more meaningful coherence throughout the generated prototype.

3.3 System Architecture and Workflow

Grounded in this theoretical framework, RPGAgent's architecture is implemented as a modular, three-stage pipeline that transforms a high-level concept into a playable prototype: (1) Narrative Generation, (2) Scene Generation, and (3) Mechanic Implementation. This hierarchical process, detailed below and illustrated in Figure 3, ensures that all generated content remains semantically aligned with the core narrative.

- **Stage 1: Narrative Generation.** The workflow begins with the Narrative Designer agent. It expands a user's natural language concept into a comprehensive narrative framework, including the world, characters, and dialogues. Critically, it segments the story into discrete, structured steps that serve as a scaffold for all subsequent design.
- **Stage 2: Scene Generation.** In this stage, the Scene Designer Agent interprets each narrative step to generate a corresponding spatial environment. It uses PCG techniques to assemble terrain and architecture from asset libraries and can invoke diffusion models to synthesize new visual elements.
- **Stage 3: Mechanic Implementation.** The final stage involves two agents Mechanics Designer and Code Generator. First, the Mechanics Designer Agent analyzes the narrative and scene to define core gameplay actions. Thereafter, the Code Generator formalizes these mechanics into executable logic, producing a working prototype where story, space, and gameplay are tightly integrated.

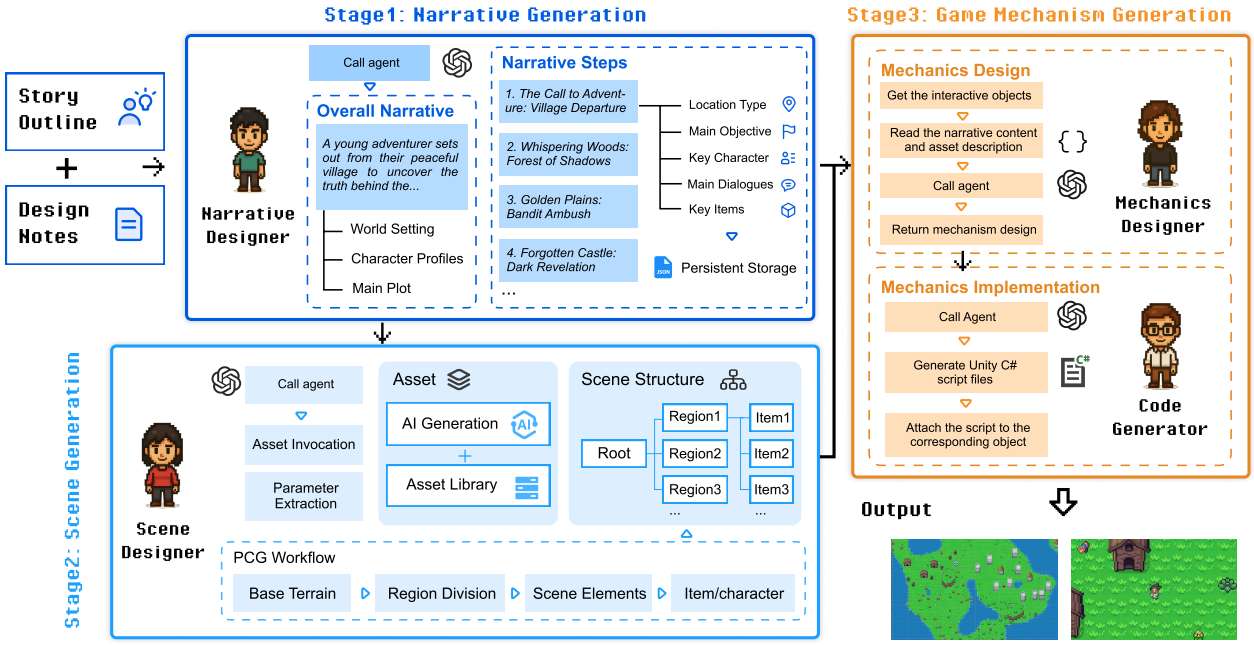


Figure 3: The modular three-stage agent workflow of RPGAgent, covering narrative generation, scene generation, and game mechanics implementation.

3.4 System Implementation

3.4.1 Development environment. The RPGAgent prototype employs a hybrid architecture, combining a Unity 6 front-end with a Python back-end. Python's robust AI ecosystem, enhanced by the Autogen framework, manages the specialized agents for narrative, scene, and mechanics, while Unity focuses on real-time rendering and executing generated content. This separation enables modular, high-performance interaction and flexible agent collaboration.

This architecture highlights RPGAgent as a co-creation system designed to assist rather than replace designers and existing workflows. By supporting early-stage ideation and rapid iteration, it generates a coherent foundation of narrative, scenes, and gameplay, while allowing designers to guide and refine the content, improving both efficiency and creative satisfaction.

3.4.2 The Multi-Agent Framework. RPGAgent employs a linear multi-agent architecture, structured around four specialized agents: the Narrative Designer Agent, which generates the foundational story and world-building elements; the Scene Designer Agent, which translates narrative contexts into spatial representations; the Mechanics Designer Agent, which defines gameplay mechanics and player interactions; and the Code Generator Agent, which transforms these specifications into executable game scripts. Each agent operates sequentially, consuming outputs from its predecessor and contributing enriched representations to a shared memory space for downstream agents, with their detailed LLM prompt templates provided in Appendix A.

This linear workflow ensures upstream outputs provide consistent foundations for downstream processes [42]. For instance, the Narrative Agent outputs descriptors for scenes and characters, which the Scene-Grounding Agent uses to generate terrain layouts and asset placements. The Mechanic Designer Agent integrates mechanics like NPC actions or environment-based challenges, which the Code Generator Agent converts into deployable scripts. By modularizing tasks, the framework maintains scalability and clear separation of responsibilities.

To handle inconsistencies, downstream agents can flag errors to their immediate predecessor for localized updates, avoiding the need for full pipeline regeneration. This combination of sequential agent execution, structured memory, and targeted revision ensures a robust, extensible system for automating and aligning the narrative, spatial, mechanical, and executable layers of game design.

3.4.3 Submodule Development and Integration. The multi-agent framework comprises four core modules-Narrative, Scene, Mechanics, and Code Generation-each handled by a specialized agent to ensure coherent and modular game design.

Narrative Generation Module. The Narrative Generation Module serves as the entry point of the RPGAgent pipeline, transforming high-level user prompts into a coherent and well-structured narrative foundation. This process is orchestrated by a Narrative Designer Agent. Upon receiving a natural language input, typically a story outlining a game concept and accompanied by optional narrative notes, the agent first derives a complete game narrative that comprises three foundational components:

- **World Setting**: A consistent fictional universe including geography, factions, cultural motifs, and key conflicts. This

serves as the narrative backdrop for spatial and mechanical design.

- **Character Profiles**: A set of major characters including the protagonist, allies, and antagonists. Each character is described with roles, motivations, emotional arcs, and potential relationships.
- **Main Plot**: A high-level narrative arc that outlines the player's overarching journey, structured into canonical story phases such as exposition, rising tension, climax, and resolution.

Once the complete game narrative is generated, the Narrative Designer Agent decomposes the main plot into a sequence of *narrative steps*, each representing a discrete episode in the player's journey. These steps are goal-oriented and defined from the player's perspective in terms of actions or tasks required to advance the story (e.g., "Rescue the village elder from the goblin cave," or "Recover the ancient artifact from the sunken temple"). This stepwise structure aligns with the logic of gameplay progression and enables modular integration with downstream systems.

To ensure consistency between the narrative logic and its spatial realization, we adopt a scene-modeling strategy that encodes temporal-spatial context, semantic annotations of interactive elements, and prototypical scene structures [4]. Each narrative step is enriched with structured semantic fields describing the primary objective, scene type, key objects, locations, main characters, and associated dialogues. These annotations serve as standardized interfaces for downstream generation modules: spatial tags inform the construction of corresponding environments, whereas character and dialogue tags provide semantic grounding for interaction logic. A representative narrative step is illustrated as follows:

```
Step Order: 1
Title: The Call to Adventure: Village Departure
Location: Village
Objective: Learn about the disappearance and prepare for
     the journey

Key Characters:
  - Kael
  - Elder Marlon

Main Dialogues:
  - Kael: "Elder, tell me everything you know about Lina's
     disappearance."
  - Elder Marlon: "She went to investigate the old ruins...
      and never returned."

Key Items:
  - Adventurer's Manual (unlocks tutorial)
  - Novice Cloak (starter equipment)
```

Listing 1: Example narrative step output

Scene Generation Module. The Scene Generation Module constructs spatial environments that closely align with narrative steps from earlier stages. It synthesizes playable and coherent spaces by operating on structured outputs from the Narrative Designer, including annotated scene types, location descriptors, object semantics, and stylistic cues. Throughout this process, the Scene Designer Agent actively interprets narrative information and orchestrates each generation step, ensuring the synthesis remains contextually grounded and adaptive to narrative intent. From semantic parsing through terrain generation to object layout, the pipeline consists of three steps, as illustrated in Figure 4.

In the first step, agent-guided PCG synthesizes diverse landscapes by stratifying the map into semantic layers—water, plains, mountains, and cliffs—which determine the assignment of specific terrain tiles. Terrain formation is driven by Perlin noise, with the agent modulating generation parameters based on narrative descriptors. Key parameters include: (1) *Noise Scale*, governing topographic roughness; (2) *Land Threshold*, which regulates the overall land-to-water ratio; (3) *Mountain Threshold*, dictating the emergence of elevated features atop land; and (4) *Island Factor*, which applies boundary falloff to shape isolated landmasses. For instance, a "remote archipelago" triggers a high *Island Factor*, whereas a "vast plain" utilizes a restrictive *Mountain Threshold* to produce expansive, flat terrain.

With the base terrain and layers established, the environment is segmented into spatial zones aligned with the narrative steps. The agent uses Voronoi diagrams structured by the narrative flow to divide the environment into distinct and semantically meaningful regions. To improve the quality and narrative fidelity of these partitions, the Voronoi generation process incorporates several optimizations: (1) Lloyd's relaxation balances zone areas by repeatedly moving each seed to the centroid of its region; (2) spatial constraints ensure seeds remain within valid terrain areas (e.g., avoiding water bodies), preserving navigability and plausibility; (3) Perlin noise perturbations are added to Voronoi edges, creating organic and visually varied boundaries.

In the second step, scene elements are distributed according to predefined configuration profiles. The Scene-Grounding Agent handles selection and placement using Unity's Assembly Definitions, which define configuration profiles for each location type. Two configuration types are supported: *Element Template*, specifying composite elements that span multiple tiles, and *Distribution Template*, which defines parameters such as element types, counts, weights, and minimum spacing (Figure 5).

The Scene Generation Agent produces placement probabilities for all candidate elements, which are then applied through the *Distribution Template* to determine their positions on the map. Each element e_i is assigned a weight w_i, either specified in the configuration profile or generated by the Agent based on design objectives such as thematic consistency, diversity, or scene-specific constraints. The placement probability is then computed as:

$$P(e_i) = \frac{w_i}{\sum_{j=1}^{|E|} w_j} \tag{1}$$

Higher weights increase the likelihood of instantiation. After probability-based selection, placements are filtered by spatial constraints, such as minimum spacing and valid terrain areas, to ensure coherent and thematically consistent layouts. As illustrated in Figure 6, by modulating generation parameters and assigning distinct asset sets, the system synthesizes diverse level from grasslands to deserts and snowfields.

In the third step, the system-under the agent's semantic guidance-refines the spatial layout by allocating key items and characters from the narrative to contextually significant locations. These placements are informed by semantic fields within each narrative step, as interpreted by the agent, ensuring that every key entity or object

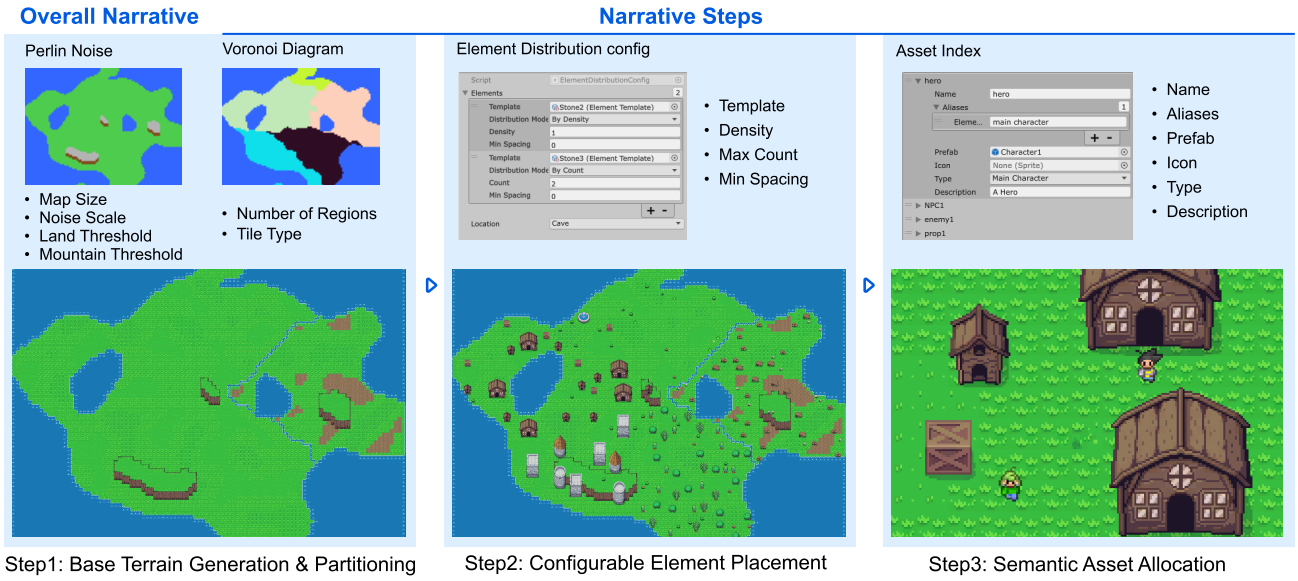


Figure 4: Scene generation process comprising three sequential steps from narrative input to spatially grounded environments.

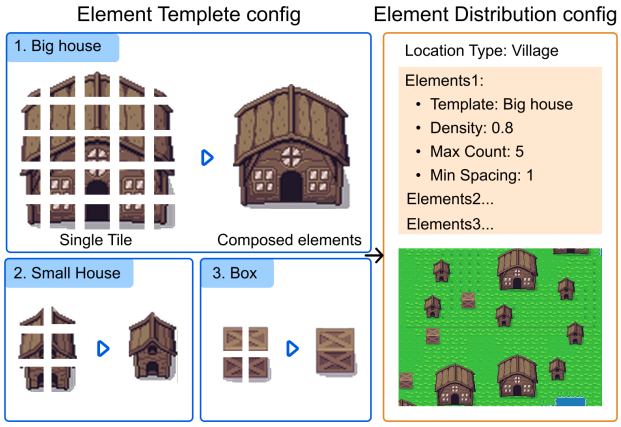


Figure 5: Two configuration types: Element Template (left) defines composite multi-tile elements; Element Distribution (right) specifies types, counts, weights, and spacing for regional placement.

reinforces both story progression and environmental affordances. For instance, a quest artifact might be positioned in a concealed cave section guarded by procedurally generated obstacles.

To facilitate the agent's selection process, an AssetIndex is defined, cataloging all available assets with their Name, Type, Aliases, and a simple description. This structured index enables the agent to quickly reference and select contextually appropriate assets for placement; for instance, a narrative descriptor for "ancient ruins" can be dynamically mapped to specific prefabs like *Broken Pillar* through semantic matching.

Art assets in the scene are obtained in two ways: by fetching assets from a curated prefab library using semantic tags specified by the Narrative Designer, and by generating missing assets with diffusion models. PCG techniques ensure density and stylistic coherence for decorative details such as rocks and foliage. Scenes are structured as semantic trees for efficient querying and dynamic updates, coordinated by the agent to support modular authoring. Through probabilistic PCG and LLM-guided control, the module links narrative abstraction with spatial realization, resulting in environments that are both diverse and narratively coherent.

Mechanics Implementation Module. The Mechanic Generation Module comprises two intelligent agents-one responsible for mechanic design and the other for code implementation-working collaboratively to bridge narrative intent and interactive realization. This module operates over structured outputs from both the Narrative Designer and the Scene Generation Module, including annotated narrative functions, character roles, player objectives, as well as detailed scene representations that specify spatial configurations and the distribution of interactable objects.

For each narrative step, the Mechanics Design agent analyzes key semantic fields such as narrative function (e.g., confrontation, discovery), object affordances (e.g., openable, collectable, hostile), object locations, and player role (e.g., investigator, rescuer, combatant). It extracts relevant entities and their spatial positions from the generated scene layout to ensure contextualized mechanic placement. Based on these inputs, the agent selects appropriate interaction types from a predefined library covering core gameplay categories such as dialogue, exploration, combat, puzzles, stealth, and item manipulation.

Rather than assigning abstract labels, the agent formulates each mechanic as a concrete *interaction specification* explicitly grounded in in-game entities. Each specification details the mechanic type,

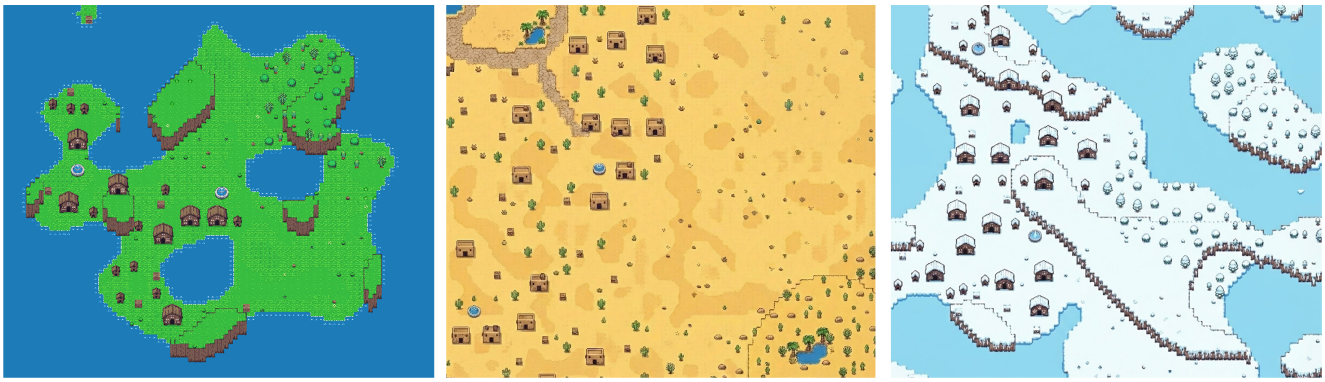


Figure 6: Visual results of scene generation across different contexts. The system adapts via specific mapping relationships between narrative semantics and PCG parameters: (a) Grassland, shaped into an island layout by increasing the *Island Factor* to enforce boundary falloff and calibrating the *Land Threshold*; (b) Desert, generated with a lower *Noise Scale* to form smooth dune patterns and an adjusted *Land Threshold* to minimize water coverage; and (c) Snowfield, characterized by a high *Noise Scale* (and *Mountain Threshold*) to create fragmented, rugged terrain. These topographic bases are populated with context-specific assets to finalize the scene.

function, interaction mode, possible outcomes, and its relationships with other game systems. These are serialized into structured JSON descriptors, serving as an intermediate abstraction between design and implementation. The structure of these specifications for interactive entities is as follows:

After the mechanic design is completed, the Code Generator Agent converts the abstract interaction specifications into executable game logic that runs within a constrained LLM-based framework. This framework is preconfigured to be highly restrictive to ensure reliability, rather than allowing the LLM to generate open-ended code.

The current prototype library is built around four main base templates: main character (player controls), NPCs, enemies, and items. These templates use an object-oriented approach, allowing specialized child templates to inherit from them. This ensures that shared logic is decoupled from specific behaviors, promoting reusability and low coupling. For instance, a base template for NPCs defines shared properties, while specialized child templates (like those for dialogue or trading) inherit from it to implement distinct interaction logic.

The agent's workflow using this library proceeds in the following two steps:

(1) Template Selection: The agent first selects a "suitable" template by mapping a mechanic type (e.g., `Dialogue`) from the Mechanics Designer's JSON specification to the corresponding C# template.
(2) Constrained Instantiation: The agent then uses the LLM to only fill the predefined placeholders in that template. The LLM parses the JSON to extract the correct entity references, event conditions, and logic blocks, and inserts them into the template.

```
Type: MainCharacter
Name: Lyla
Mechanics:
  Function: Player-controlled character who can explore the
      environment, collect items, and interact with NPCs.
  InteractionMode: Keyboard and mouse input
  Outcome: Gains experience and unlocks new abilities when
     completing quests
  SystemRelationships: Connected to combat, dialogue, and
     exploration systems

Type: NPC
Name: Elder Rowan
Mechanics:
  Function: Provides quests, hints, and storyline dialogue
     to the player
  InteractionMode: Dialogue triggered by player proximity
  Outcome: Relationship level increases after quest
     completion, unlocking new story branches
  SystemRelationships: Linked to quest system, shop, and
     narrative branching

Type: Item
Name: Ancient Amulet
Mechanics:
  Function: A rare artifact that unlocks access to the
     hidden temple when carried
  InteractionMode: Collected via exploration or as a quest
     reward
  Outcome: Consumed upon use to open the temple entrance,
     enabling new story progression
  SystemRelationships: Tied to inventory system, quest
     logic, and environmental triggers
```

Listing 2: Example Interaction Specification

For example, if the Mechanics Designer's JSON specifies a `Combat` mechanic for an `Enemy` asset (e.g., a `Forest Troll`), the agent selects the appropriate enemy combat template (which inherits from

the base enemy template). The LLM's sole task is then to instantiate this template by filling its predefined placeholders, such as setting `health` to `50`, `attackDamage` to `5`, and `aggroRadius` to `10`.

This "fill-in-the-blanks" approach, combined with predefined structural constraints, guarantees that the generated code is syntactically correct and consistent, and supports a wide variety of gameplay mechanics. The C# scripts are then attached to the corresponding prefab assets in the scene. This completes the end-to-end mechanic generation process, from high-level narrative intent and semantic analysis, through mechanic specification design, to concrete implementation in the game engine, as illustrated in Figure 7.

To ensure functionality, a technical verification layer automatically evaluates all instantiated C# scripts. This verifier performs concrete checks for reference integrity (e.g., cross-referencing entity names and item IDs against an asset database), parameter validation (e.g., matching instantiated values to the template's required data types), and dependency checking. If any inconsistencies are identified (e.g., a non-existent item ID), this layer automatically triggers a targeted feedback loop: a structured error message is sent back to the Code Generator Agent (e.g., "Error: Reference not found"), prompting it to regenerate a corrected script. This iterative process continues until all scripts pass verification, ensuring that only valid, executable code is integrated into the final game prototype.

3.4.4 System Integration and User Interaction. To enable real-time co-creation between LLM-based agents and game designers, RPGAgent adopts a hybrid architecture that integrates a Unity visual editor as the frontend with a Python multi-agent backend powered by Flask. The Unity editor provides intuitive interfaces for user input, and scene visualization, serving as the primary point of interaction and feedback for designers.

As shown in Fig. 8, the frontend and backend communicate asynchronously. Within the Unity client, user input is first serialized into JSON before being dispatched as an HTTP request to the Python server. On the backend, the request is routed by the Flask framework, after which specialized agent instances are created to process the data. These agents then send API calls, again in JSON format, to our LLM endpoints.

Our system utilizes OpenAI's GPT-4o (gpt-4o-2024-08-06). To guarantee strict adherence to complex output schemas, we applied Supervised Fine-tuning (SFT) to the rule-heavy Scene and Narrative Designer Agent. Given that these agents operate under extensive constraints (system prompts > 100 lines), we fine-tuned them using a high-quality, hand-crafted dataset ($N = 80$). This process was designed specifically to enforce syntactic correctness and structural integrity, ensuring that outputs rigorously comply with the game engine's logic rather than injecting new knowledge. While simpler agents utilize the base model, this targeted optimization ensures that complex generated content can be reliably deserialized by the Unity client for immediate visualization.

The modular microservice design of the backend allows each agent to function independently, with intermediate results easily exchanged or returned for real-time visualization and designer inspection. This architecture supports iterative workflows: designers can adjust scene parameters (e.g., terrain complexity, mechanic difficulty) via Unity's UI controls (Figure 9), triggering regenerated prompts and updated outputs that are seamlessly reflected in both the backend state and the visual scene.

To ensure robustness, the system employs layered validation and error-checking mechanisms at key stages of the workflow. Backend responses undergo structural validation using JSON Schema to ensure compliance with expected data hierarchies, while semantic checks verify domain-specific constraints, such as range limits for parameters like terrain complexity or mechanic difficulty. In cases of incomplete or conflicting data, fallback mechanisms assign values within acceptable ranges to maintain workflow continuity.

4 User Study

To explore how a multi-agent system supports creativity, user experience, and interaction patterns in early-stage game design, we conducted an exploratory study with 18 participants using the RPGAgent platform, treating quantitative results as indicative trends and complementing them with qualitative analysis of co-creation processes. The study focuses on how users engage with agent-generated content, how creative ideas are sparked through interaction, and how narrative, spatial, and gameplay elements evolve through collaborative authoring.

4.1 Participants

A total of 18 participants (10 male, 8 female; aged 20–29, $M = 23.7$) were recruited for the user study. Most were familiar with LLM tools, though only a few had advanced proficiency with game development engines, as detailed in 5.2. Participants were either undergraduate or graduate students in art, architecture, design, computer science, or related fields. All participants volunteered to take part in the study and provided informed consent in accordance with institutional ethics guidelines. The participant pool was intentionally balanced in terms of gender and design background to support both qualitative feedback and quantitative comparison between procedural and manual scene design workflows.

4.2 Experimental Design and Procedure

The study employed a within-subjects crossover design to evaluate how a multi-agent system supports creative game design processes and prototyping experiences compared to a baseline workflow. All 18 participants completed both conditionsâĂŤusing RPGAgent and the baseline workflowâĂŤallowing us to assess design efficiency, content coherence, and creative satisfaction while accounting for individual differences. To mitigate potential order effects, the experimental order was counterbalanced: nine participants were randomly assigned to use RPGAgent first, while the other nine began with the baseline workflow.

To ensure fairness, both workflows provided participants with identical sets of art assets and required the use of Unity's Tilemap feature for level construction. In the RPGAgent condition, participants utilized the integrated agent-driven "story-to-play" pipeline. Conversely, the baseline condition required manual creation with standard authoring tools (Unity Tilemap). Participants were allowed to consult GPT-4o during the baseline condition for narrative writing or coding tasks; however, all integration and implementation had to be performed manually. This setup simulated a

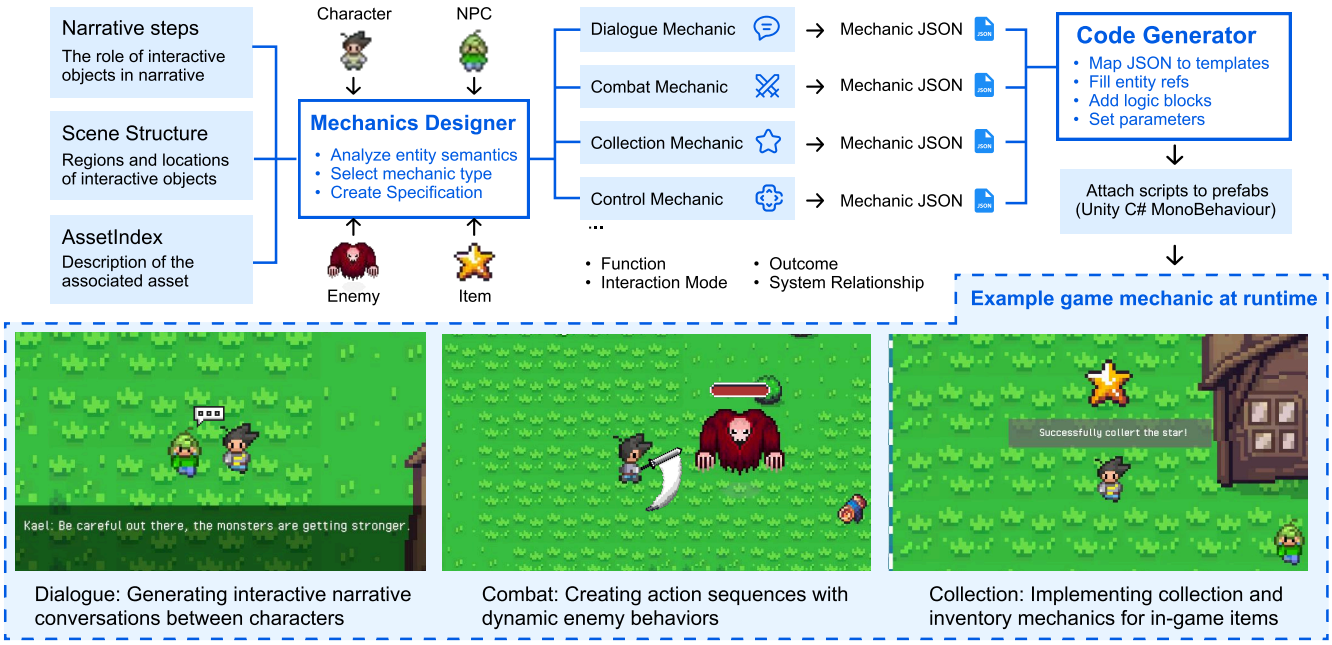


Figure 7: Workflow for implementing game mechanics and example mechanic outputs rendered in the game engine at runtime.

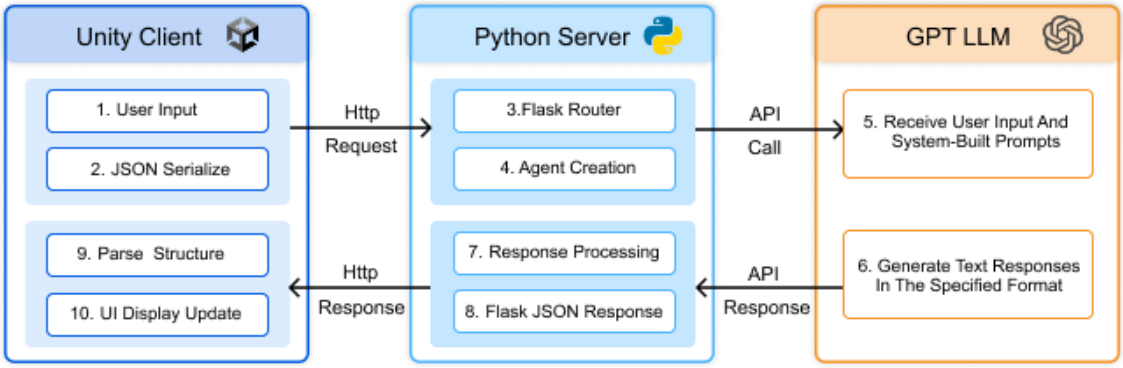


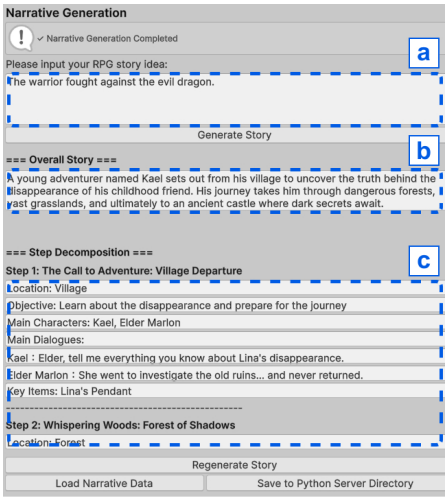
Figure 8: Architecture and data flow of RPGAgent's Unity-Python-LLM Communication.

non-integrated workflow where LLM tools assist creativity but are not deeply embedded within the engine.

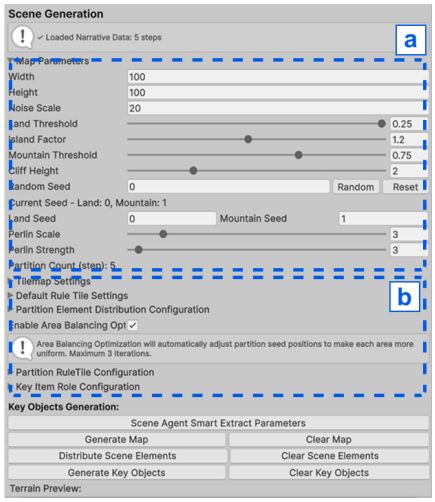
The design task required participants to create a short, interactive scene set in a fictional forest village. Participants were free to define their own game story outline, allowing for a wide range of creative interpretations while maintaining a consistent basis for comparison. The study began with a pre-session intake survey, followed by task introduction and training (5 minutes per tool, 10 minutes total). During the main task phase (20 minutes per condition), participant actions were logged, and a think-aloud protocol was used to capture their reasoning and decision-making processes.

Data collection captured interaction logs tracking agent suggestions, edits, and manual operations in both workflows. After completing the design tasks, participants filled out a post-task survey (approximately 15 minutes), followed by a semi-structured interview (approximately 20 minutes). Quantitative data were collected through 7-point Likert scale instruments. Creativity items were adapted from the Creativity Support Index (CSI) [11], with collaboration-related items removed, as the study focused on individual workflows. Experience items were drawn from the User Experience Questionnaire (UEQ) [36], assessing participants' perceptions of usability, engagement, and satisfaction during each workflow.

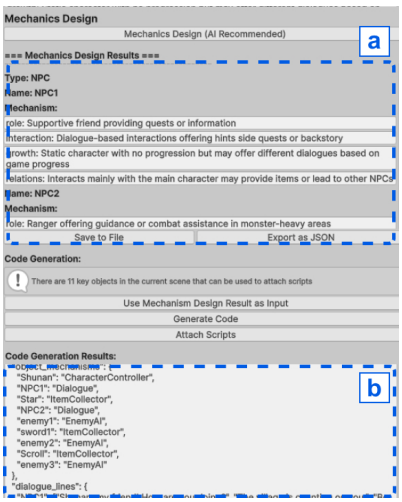
In addition to quantitative measures, participants took part in a semi-structured interview at the end of the session to explore qualitative aspects of user experience. These interviews focused on key topics: (1) comparisons between RPGAgent and the baseline



(a) Narrative Tab: (a) User input area for story outline, (b) Display of the expanded complete game storyline, (c) Display of detailed narrative steps.



(b) Scene Tab: (a) Configuration of scene parameters, such as layouts and visual effects, (b) Scene presets for quick setup of environments.



(c) Mechanics Tab: (a) Tools for designing game mechanics, (b) Code preview panel for inspecting the generated scripts.

Figure 9: The user interface of RPGAgent integrated into the Unity Editor. It consists of three primary tabs: Narrative, Scene, and Mechanics, each tailored to streamline different aspects of the game design process.

workflow, (2) perceptions of whether the generated content was coherent and aligned, and (3) recommendations for improving either workflow. This qualitative feedback provided deeper insights into user perceptions, highlighting the strengths and limitations of both workflows while offering suggestions for future enhancements.

4.3 Data Collection and Analysis

We collected multimodal data to capture a holistic view of the design process. In addition to intake and exit surveys, each session was observed in person by a researcher who took detailed field notes on participant behaviors and verbal reactions. With informed consent, all sessions were audio- and screen-recorded to provide ground truth for the qualitative accounts. RPGAgent also automatically logged fine-grained agent-user interactions, including prompts, responses, manual edits, and intermediate design states for both conditions.

For quantitative data, we employed the Wilcoxon signed-rank test to compare RPGAgent with the baseline workflow. This non-parametric test was selected as it is well-suited for our within-subjects design with a sample size of $N = 18$ and accounts for the ordinal nature of the UEQ and CSI scales. To control the Family-Wise Error Rate (FWER) arising from multiple comparisons across the sub-dimensions of the surveys, we applied the Holm-Bonferroni correction to all reported p-values. Additionally, we conducted a post-hoc power analysis using G*Power to provide a reflexive assessment of our statistical sensitivity. Given the large effect sizes observed in our primary metrics—specifically the overall CSI ($d_z = 2.74$) and overall UEQ ($d_z = 2.46$) scores—the statistical power ($1-\beta$) reached 0.99 for all composite measures. For the sub-dimensions where significant differences were found (e.g., Efficiency, Enjoyment, and Exploration), the power ranged from 0.86 to 0.99, confirming that our study had sufficient sensitivity to detect the substantial impacts of RPGAgent despite the small sample size.

For our qualitative analysis, we followed a reflexive thematic analysis approach [8, 9]. This analysis was informed by our complete multimodal dataset. Audio recordings were transcribed, and this data was systematically triangulated with screen recordings and interaction logs to reconstruct participants' decision-making processes. Following the reflexive paradigm, two researchers engaged in an iterative coding process supported by 3 collaborative analysis meetings over 2 weeks to review codes, challenge interpretations, and refine candidate themes. In the first round of coding, we generated 64 raw codes, which were clustered into 12 candidate topics; through iterative merging/splitting and refinement grounded in the multimodal triangulation, these topics were consolidated into the 6 final themes presented in Section 5.3. Rather than seeking inter-coder reliability through quantitative agreement statistics, we utilized intensive consensus discussions to refine the themes, ensuring they captured the nuances of agent negotiation, control delegation, and narrative coherence.

5 Results

5.1 Qualitative System Observations

We experimented with various interaction protocols, most notably the structure of the data handoff between the Narrative and Scene agents. We note that when using an unstructured natural language stream, the system will not always specify concrete content for each stage. In simpler scenarios, like "a hero enters a village," it often generates immersive story text but fails to output the discrete spatial descriptors required by downstream agents. To remedy this,

we enforced a rigid Narrative Step JSON contract, forcing the system to explicitly discretize the story into executable steps.

With scene parameterization, we focused on the mapping reliability for hyperparameters such as `noiseScale` $\in [10, 50]$ and `landThreshold` $\in [0.1, 0.4]$. We observed that the system often struggles to adhere to strict boundary constraints. For example, when prompting for an "archipelago," the system would frequently drift the `landThreshold` towards 0.5, ignoring the required range of $[0.10, 0.25]$ necessary for island formation. This speaks to a problem we noticed occasionally where the agent prioritizes semantic associations over numerical precision. To remedy this, we applied SFT to the Scene Agent, implicitly encoding these boundary rules into its weights. Furthermore, we noted anecdotally that the system sometimes attempts to place assets not present in the `AssetIndex`, such as calling for specific "tropical palms" when only generic "oak trees" are available. We adapted our prompts to insist that the system restrict its selection strictly to the provided prefab library.

Regarding code generation, we observed a tendency for the system to invent tasks beyond the capability of the submodules to which it has access. For instance, when asked for a "stealth mechanic," it would sometimes attempt to implement complex raycasting logic that conflicted with the existing character controller. To remedy this, we restricted the coding module to a "template instantiation" mode. Instead of writing raw C# classes, the agent is constrained to filling placeholders in pre-validated `MonoBehaviour` templates. We note that allowing the code generation submodule to modify logic freely almost always resulted in compilation errors due to missing references. In contrast, the template approach allows for small but observable functional changes—such as modifying interaction parameters or event triggers—without breaking the build.

Collectively, these observations highlight the inherent conflict between divergent creativity and convergent precision. Narrative ideation naturally thrives on unpredictability, whereas technical implementation requires exact code syntax and strictly bounded PCG parameters. A single-agent system often struggles to reconcile these opposing objectives within a single context. RPGAgent resolves this through Role-Based Isolation, decoupling the "creative writer" from the "technical implementer" to ensure that narrative flexibility does not compromise structural reliability.

5.2 Quantitative Results

Analysis of the quantitative data collected from the study indicates that RPGAgent, through its multi-agent framework, effectively enhances the user experience and facilitates the generation of creative ideas. It supported participants in identifying relevant references and integrating their own creative contributions into the RPG prototype design and development process.

The pre-questionnaire results revealed that nearly all participants possessed at least some familiarity with LLMs, whereas their familiarity with game design tools showed greater variability. In particular, as depicted in Figure 10, most participants reported limited or no prior experience with Unity. The limited prior experience with Unity and game development among participants provided a relevant foundation for analyzing how RPGAgent supports novices, offering insights into its role in lowering barriers and shaping early creative experiences.

The UEQ results (detailed in Table 1 and visualized in Figure 11) demonstrate that RPGAgent significantly outperformed the baseline workflow across most dimensions. Participants rated RPGAgent as significantly more *Attractive* ($p < .01$) and *Perspicuous* ($p < .05$). The most substantial advantages were observed in dimensions related to the tool's generative and exploratory nature: *Efficiency*, *Stimulation*, and *Novelty* (all $p < .01$), reflecting the system's capacity to rapidly generate playable prototypes while sparking user engagement through novel, inspiring content.

As illustrated in Figure 11, RPGAgent's scores for *Attractiveness* and *Efficiency* also exhibited narrower interquartile ranges (IQRs), indicating greater agreement among participants on its positive impact. In contrast, *Dependability* was the only dimension with no significant difference ($p = .125$) and showed higher variability. This suggests divergent perceptions of the system's reliability, potentially attributed to the randomness introduced by procedural generation. This aspect will be discussed in detail in Section 6.1. Cumulatively, RPGAgent achieved a significantly higher *Overall UEQ score* ($p < .001$), reinforcing its superior user experience

The CSI results offer a more nuanced view of how RPGAgent supports the creative process (Table 1 and Figure 12). Participants rated RPGAgent significantly higher for *Exploration* ($p < .01$), highlighting the tool's ability to facilitate creative discovery. Similarly, RPGAgent was found to be substantially more *Enjoyable* ($p < .001$), reflecting greater engagement.

In terms of creative depth, no statistically significant differences were observed for *Expressiveness* ($p = .425$), *Immersion* ($p = .408$), or *Effort/Reward* ($p = .078$). As shown in Figure 12, the distributions for these dimensions are relatively similar between the two conditions. While the system showed a positive trend in *Effort/Reward*, it did not reach statistical significance after the Holm-Bonferroni correction. Together with the significant gains in UEQ Efficiency, this suggests that RPGAgent lowers the barrier to entry while maintaining expressive capability and immersion comparable to manual workflows.

Cumulatively, RPGAgent achieved a significantly higher *Overall CSI score* ($p < .001$), indicating that it provides superior holistic support for creativity. The narrower IQRs and higher medians for *Exploration* and *Enjoyment* (visible in Figure 12) underscore RPGAgent's role in enhancing the creative process while ensuring a consistent positive experience for participants.

5.3 Qualitative Feedback

Participants provided diverse insights into the system's utility, framed by their individual backgrounds and levels of experience with game development tools. Our analysis identified six critical themes.

Creative Affordances and Scaffolding. Participants consistently praised the system's ability to spark new ideas and accelerate prototyping. We observed this synergy directly: a simple prompt like "a missing child in a mountain village" (P2) was dynamically expanded into multi-step dialogue arcs, corresponding spatial layouts, and interactable logic, all aligned with the narrative's progression. This confirmed the system's role as a "creative scaffold." For instance,

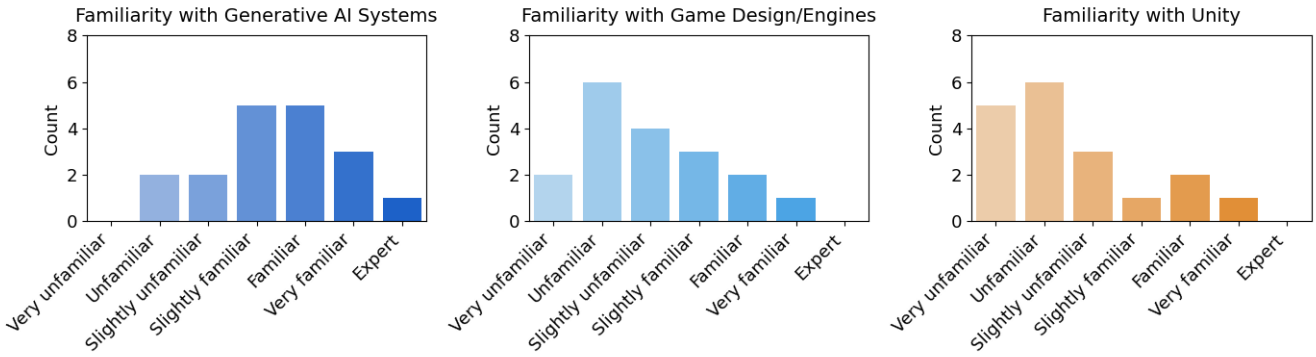


Figure 10: Self-reported familiarity with LLMs, game design tools, and Unity Editor.

Table 1: Survey results of participant opinion from the UEQ and CSI (N=18). Mean (M) and Standard Deviation (SD) are reported. Reported p-values are adjusted using the Holm-Bonferroni procedure.

Indicator		RPGAgent		Baseline		Adj. p-value
		Mean	SD	Mean	SD	
User Experience Questionnaire	Attractiveness	5.119	0.489	4.286	0.871	0.00351**
	Perspicuity	5.111	0.564	4.458	0.649	0.02308*
	Efficiency	5.236	0.565	3.278	0.499	0.00114**
	Dependability	4.292	0.837	4.514	0.539	0.12512
	Stimulation	5.314	0.491	3.426	0.981	0.00114**
	Novelty	5.208	0.479	3.056	0.532	0.00114**
	Overall UEQ score	**5.047**	**0.223**	**3.836**	**0.498**	**0.00004***
Creativity Support Index	Exploration	5.278	0.492	4.250	0.912	0.00636**
	Expressiveness	5.278	0.669	5.139	0.660	0.42452
	Immersion	4.583	0.691	4.389	0.796	0.40758
	Enjoyment	5.722	0.521	3.528	1.021	0.00095***
	Effort/Reward	5.306	0.730	4.722	0.647	0.07755
	Overall CSI score	**5.233**	**0.325**	**4.406**	**0.458**	**0.00019***

Note: p-values for UEQ sub-dimensions ($m = 6$), CSI sub-dimensions ($m = 5$), and overall scores ($m = 2$) were separately adjusted using the Holm-Bonferroni procedure to control for FWER. Significance: *p<0.05, **p<0.01, ***p<0.001.

P1 (with prior development experience) remarked, "I don't need the system to finish my game for me, but it is great at giving me a starting point; it feels like having a junior designer on the team." In contrast, P6 (LLM user, new to game dev) commented, "The agents create branching stories and layouts that are both surprising and helpful, which makes me reconsider how a scene could unfold."

Similarly, interaction logs revealed instances where procedural quirks inspired narrative pivots. P18 initially prompted for a generic village setting without specifying terrain features. However, the Scene Designer's randomized noise parameters generated a massive lake dominating the map center (Figure 13a). This unexpected geographical feature caught P18's attention; they remarked, "I hadn't planned for water, but this looks distinctive," and subsequently pivoted the narrative from a standard hamlet to a "lakeside sanctuary," adapting the lore to the generated topology. Figure 13 further illustrates the range of landforms produced by lightweight prompts, and how emergent features can provide a concrete scaffold for narrative adaptation.

Observed Creative Boundaries. The study confirmed the system's creative boundaries (as noted by R3), revealing a recurring tendency toward narrative homogenization. In particular, the Narrative Designer often re-framed unconventional prompts into familiar RPG quest structures. This was evident in P5's interaction log: the non-RPG prompt "a monkey searching for bananas" was transformed into a conventional three-act quest (e.g., "In a tropical forest, the little monkey hears of a mysterious golden banana ... must cross the dangerous forest ... finally finding the golden banana tree."). The resulting JSON further codified this framing by mapping `monkey` to `MainCharacter` and `golden banana` to `keyItem`. Downstream, the Scene Designer could not realize the intended concept due to missing assets, substituting a generic placeholder instead (Figure 14).

A similar loss of specificity occurred with P9's prompt, "a whispering tree that remembers." The Narrative Designer produced a

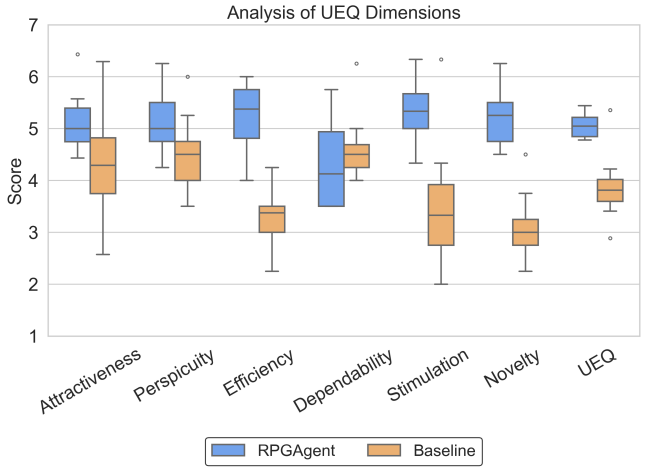


Figure 11: Distribution of UEQ scores for participants. Boxes represent IQRs; whiskers extend to 1.5× IQR; dots indicate outliers.

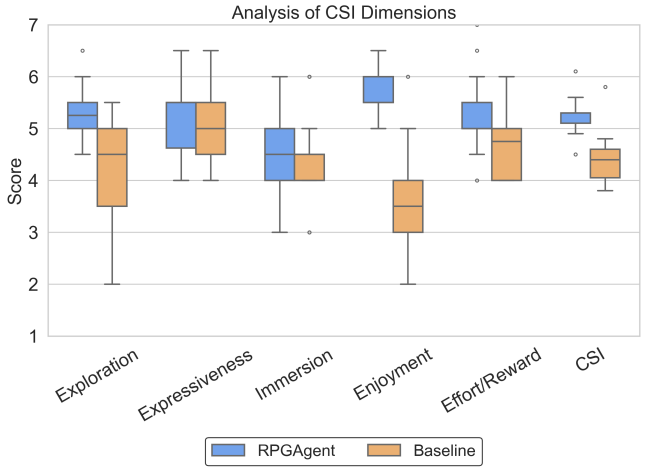


Figure 12: Distribution of CSI scores for participants. Boxes represent IQRs; whiskers extend to 1.5× IQR; dots indicate outliers.



(a) Plainland with an emergent lake. *Prompt:* "plain".



(b) Fragmented archipelago. *Prompt:* "many islands".



(c) Single island settlement. *Prompt:* "island".



(d) Mountainous terrain. *Prompt:* "mountain".

Figure 13: Prompted terrain variants with an emergent lake feature. Across four representative terrain prompts, the system produces diverse landforms; notably, an unexpectedly large lake can emerge even under a plainland prompt.

poetic quest framing (e.g., "In an ancient forest, a whispering tree remembers past events …"), but the Scene Designer did not translate the intended aesthetic into an appropriate visual artifact. Rather than introducing a distinctive asset, it instantiated a standard tree from the prefab library, diminishing the prompt's magical quality. Together, these cases suggest that RPGAgent's creative boundaries



Figure 14: P5: Asset-driven placeholder and character substitution. Because no `golden banana`-related assets were available, the system automatically generated a generic placeholder to represent the intended key item. Similarly, the `monkey` was instantiated using a generic human character asset, illustrating an asset-library boundary that can induce narrative homogenization.

are shaped by both its RPG-genre priors and the affordances of the available asset library, which can constrain users' original intent when prompts fall outside these defaults.

Transparency and Task Articulation. Participants valued the system's transparency for understanding and control. P6 (novice coder) appreciated the clear intermediate outputs: "I don't need to understand all the underlying details, but being able to see and tweak each stage makes me feel more confident." P11 echoed this, stating that "tracking the generation results at each step directly within the Unity interface allowed me to immediately see and validate the output." However, this transparency also highlighted the system's complexity for expert users. For instance, P7 (very familiar with Unity), who wanted to write code by hand to implement new features, found they first "had to understand the original inheritance structure" of the generated scripts. P7 described this reverse-engineering process as "sometimes overwhelming" and cited difficulties in detecting embedded errors. To streamline debugging, P7 suggested implementing an "error summary log" with clear references to relevant code sections, allowing users to "intervene without wasting time chasing minor details."

Interaction Pattern: Bridging Semantic–Spatial Gaps. We observed a recurring interaction pattern where participants actively managed a tension between the system's semantic comprehension and its lack of precise spatial control. The system could understand a narrative step (e.g., "a warrior arrives at the village," P2), but often failed to place the arrival point logically (e.g., placing it on the outskirts instead of a central plaza). In these instances, we observed participants seamlessly transitioning from co-creator to director. As P2 noted, "The AI got the idea, but I needed to fix where it happened." P13 faced a similar issue when generating a "crowded village." Due to the system's fixed constraints on the maximum number of NPCs per region, the generated population was too sparse to convey "crowdedness." The logs recorded the user manually dragging additional NPC prefabs from the asset library into the scene to supplement the count. Participants did not view this as a failure, but as a "well-mitigated" part of the workflow, using the Unity editor's UI to manually refine the agent-generated content. This highlights a fluid blending of automated generation with fine-grained human control.

Interaction Preferences: Iteration vs. Automation. Participants showed two distinct preferences for interaction. The majority valued the iterative, modular pipeline for its balance of creativity and control. This group, including P1, wanted to review and adjust outputs at each stage. P1 noted, "This approach, where I can control and modify the content at each step, ensures the final result aligns with my vision." This sentiment was echoed by P4 (familiar with Unity), who valued the ability to "pinpoint the step in the plan and adjust it—no need to dive deep into code changes." This iterative approach enabled deliberate refinement while maintaining alignment with user goals.

Conversely, a smaller group prioritized rapid exploration and preferred a more autonomous mode for faster workflows. P8 explained, "I prefer generating results by quickly changing the random seed—it's an easy way to explore different outcomes without spending time on step-by-step adjustments." These users saw value in a fully automated "one-shot" option to rapidly produce draft content, particularly for projects with tight time constraints. This suggests a clear need for both iterative, high-control modes and automated, high-speed modes to accommodate diverse workflow demands.

Usability Across Skill Levels. Across skill levels, participants appreciated the direct integration of procedural scene generation into the game editor, which eliminated tool-switching and kept them "in the flow." This benefit was most pronounced for our expert user. P7 was the only participant observed leveraging the system's scene parameterization features with precision. P7 described the tool as a way to "fine-tune without leaving the creative zone," treating the agent's output not as a final, rigid product, but as a malleable starting point for immediate, granular refinement within the editor.

In sharp contrast, the same parameters posed a significant usability barrier for novice users. P3 (novice) expressed this frustration clearly: "There are so many parameters . . . I still want to tweak a few things myself, and then I can't get it back to the original settings." This sentiment was echoed by P6 and P9, who, lacking a mental model of the PCG system, were hesitant to adjust the toggles and sliders. Their concern was less about experimentation and more about the lack of "safe experimentation" tools, like a reset button. This suggests that while parameters are necessary for flexibility, they create a high barrier for novices and must be layered or guided to lower the entry barrier.

6 Discussion

6.1 Impact on Design Efficiency and Usability

Our investigation into RQ1 revealed that the primary perceived benefit of RPGAgent stemmed from its nature as an integrated, in-editor workflow. Qualitative analysis showed participants consistently attributed the system's advantages to this tight integration rather than the underlying AI's generative capability. These findings align

with Creativity Support Tools principles regarding reduced friction in creative tasks. Unlike previous systems focusing on isolated tasks, such as Word2World [48] for scenes or StoryVerse [78] for narrative, RPGAgent bridges the gap between abstract storytelling and playable realization. This demonstrates that workflow continuity can be as impactful as the intelligence of individual generative models.

This feedback became especially sharp when participants contrasted RPGAgent with the baseline. They described the baseline as a "fragmented" manual workflow that required frequent tool switching and forced them to resolve inconsistencies between GPT-4o and Unity. In contrast, RPGAgent automated this integration pipeline. While the perceived efficiency largely stems from tighter integration, enabling such a seamless workflow depends on our multi-agent architecture (see system observations in Section 5.1). By isolating tasks across agents, RPGAgent produces structured intermediate artifacts that reduce schema violations and cross-stage inconsistencies that commonly arise in monolithic, single-context prompting. As a result, designers could focus on creative decisions rather than data porting and debugging, which helps explain the higher perceived efficiency.

However, higher efficiency did not translate into uniform reliability. Quantitatively, *Dependability* was the only UEQ dimension without a significant difference ($p = .125$) and showed high variability. Qualitatively, this appears driven less by integration failures than by unpredictability in generative steps. We observed two recurring issues. First, limited granularity of control: participants found the system effective for coherent high-level prototypes but less dependable for precise, fine-grained edits (e.g., adjusting small details or debugging generated code). Second, semantic asset matching problems: the Scene Agent sometimes failed to retrieve suitable assets for narrative items such as a "Forest Amulet," favoring existing prefabs over diffusion-based generation even when the match was semantically weak. Together, these issues help explain the variability in perceived dependability. These findings align with creativity support research suggesting that end-to-end integration and actionable intermediate representations can be as impactful as model quality for sustaining designers' momentum [60].

6.2 Enhancing the Creative Experience

Regarding RQ2, our study found that RPGAgent enhanced the creative experience by acting as a "creative scaffold," particularly for open-ended tasks. This enhancement manifested in three primary ways. First, the system enabled rapid prototyping; it automated the laborious setup of a blank project, allowing participants to quickly generate and iterate on multiple interactive concepts. Second, the agent-generated content itself served as a direct source of inspiration, leading to unexpected outcomes and sparking new design directions. Third, by automating the integration of narrative, scenes, and code, the system produced a tangible, visualized, and playable prototype, not just abstract text. This contrasts sharply with standard generative tools that leave implementation to the user. This automated bridge from high-level ideation to an interactive result explains the significantly higher CSI scores for *Exploration* and *Enjoyment*.

However, this enhancement is defined by a critical trade-off. Our CSI data reveals this divergence: while *Exploration* and *Enjoyment* significantly improved, *Expressiveness* showed no difference. This quantitative finding is directly corroborated by the qualitative failures of 'narrative homogenization' detailed in 5.3. Our findings thus integrate insights from co-creative systems [17] and Creativity Support Tools [60]. We empirically show that RPGAgent, acting as an AI partner, exhibits the classic Creativity Support Tools trade-off: it streamlines *Exploration* precisely by constraining *Expressiveness*. We hypothesize that this behavior may be an emergent property of a "cascade of constraints," where the narrative agent's domain biases propagate to and constrain the downstream scene agent. This suggests the next frontier is not just optimizing generation, but designing for negotiation—allowing users to inspect, challenge, and override these inherent system biases.

6.3 The Tension Between Automation and Designer Agency

Addressing RQ3, we found designers perceive the system as a "co-creative partner." We suggest this perception stems from the agent moving beyond a passive tool role to actively contribute generative content. This partnership enables a high-level "director" mode, where the designer's role shifts from direct manual creation to delegating initial ideation (the "first draft" of content) and technical implementation to the agent. Our qualitative findings support this framing: participants valued the agent-generated starting framework. Participants with prior development experience regarded it as an assistant for initial creation, while novices valued the generated content as a source of new ideas.

Beyond high-level direction, we identified a "mid-level supervision" mode, a departure from the baseline "black box" interaction typical of many generative models. RPGAgent deconstructs generation into a transparent, modular workflow, enabling two key observed behaviors. The first was *direct procedural manipulation*, where designers fine-tuned exposed parameters by adjusting Unity inspector values to guide results. The second was *iterative refinement*, which involved inspecting and re-running isolated pipeline stages when unsatisfied. By making parameters and their outcomes visible, this negotiable workflow allows designers to apply expertise at specific intervention points, shifting interaction from wholesale acceptance or rejection toward more structured control.

However, this empowerment also revealed a tension between automation and designer agency that RQ3 explores. Our findings reflect concerns that providing a "ready-made game" may hinder novice users from learning game design principles. In RPGAgent, we attempted to mitigate this by enabling manual intervention at each stage and exposing structured intermediate outputs. Still, we observed a divide in control: experienced participants leveraged the exposed parameters to "fine-tune" the output, whereas less experienced users expressed frustration with the number of options and felt unable to use them effectively. This suggests that while the tool can empower those with existing knowledge, it may overwhelm novices who lack the skills to "go back to manual edits." Future work should explore how to bridge this gap, for example by designing such systems not only as productivity tools but also as educational scaffolds that progressively expose underlying logic and support

"safe" experimentation. This points to a design implication consistent with prior co-creative systems: increasing agency may require interfaces for contesting constraints rather than merely improving the generator [17].

6.4 Positioning and Critical Reflection

We present RPGAgent as a multi-agent framework that bridges the gap between abstract creative concepts and coherent, playable prototypes. Grounded in the Elemental Tetrad, our architecture ensures that story, aesthetics, and mechanics evolve holistically. This distinguishes our approach from systems like Word2World [48] or CrawLLM [85], which primarily focus on level generation or the placement of visual elements. To achieve this cohesion, RPGAgent employs a constrained, cascading pipeline where an LLM-generated structured narrative serves as the formal "single source of truth" to orchestrate all subsequent generation. Unlike the open-ended planning of systems like [14], we prioritize output reliability over planning uncertainty by using narrative scaffolds to drive rule-based PCG and template-based coding.

This "artifact-centric" focus also positions our contribution from prior multi-agent frameworks, such as GameGPT [10] or ChatDev [55]. While those systems are process-centric—simulating software engineering roles (e.g., "programmer," "tester") to automate development tasks—our agents are functional modules designed to directly generate the creative assets themselves. This distinction is important: the goal of our system is not to manage a development process, but to ensure the artifact's coherence by holistically linking the core components of game design theory. This approach reframes the system as a co-creative partner, functioning as a creativity support tool for designers rather than an autonomous developer.

This positioning reveals an inherent trade-off: the very constraints that guarantee coherence can simultaneously inhibit expressive freedom. By enforcing a structured narrative scaffold and pre-validated code templates, RPGAgent effectively eliminates the "blank page" problem, a benefit reflected in the high *Exploration* scores for free-form tasks. However, this efficacy relies on the user's willingness to cede structural control. In convergent, fixed-target tasks, where designers approach the system with a rigid, pre-conceived vision, these same scaffolds risk becoming obstructions. The structural friction documented in our qualitative analysis of Observed Creative Boundaries serves as a prelude to this limitation. Consequently, the next evolutionary step for such frameworks extends beyond better generation to adaptive constraint relaxation, empowering designers to selectively dismantle the scaffold to prioritize expressiveness over automation.

6.5 Limitations and Future Work

While RPGAgent demonstrates a promising approach, we acknowledge several limitations that frame our future research agenda.

First, regarding generalizability, the framework currently relies on a strict narrative-first dependency structure. This assumption effectively streamlines the prototyping of story-driven RPGs, but it may not transfer to mechanics-driven genres (e.g., puzzle or strategy games), where gameplay rules often precede narrative framing. Future work will explore more flexible entry points by decoupling agent dependencies, enabling designers to start from mechanics, spatial constraints, or interaction goals and subsequently synthesize narrative elements to maintain coherence.

Second, regarding technical constraints, the Code Generator operates via constrained template instantiation rather than open-ended code synthesis. While this design improves robustness and reduces the risk of non-compilable or unsafe outputs, it also limits the space of mechanics to recombinations of existing primitives. Future iterations could integrate retrieval-augmented generation (RAG) with stronger validation (e.g., schema checking, unit tests, or lightweight formal specifications) to enable more expressive script generation while maintaining reliability.

Third, regarding study scope and validity, our evaluation prioritized comparing overall utility against a manual baseline within a short, single-session protocol. This introduces several trade-offs. (i) The baseline does not isolate the specific contribution of the multi-agent decomposition relative to a monolithic LLM pipeline; future work should include ablation-style baselines (e.g., single-agent or non-stage-revealing variants) to attribute benefits more precisely. (ii) Our participants were primarily university students with design/computing backgrounds, which may not represent truly novice creators; future studies should recruit more diverse and representative samples across expertise levels and contexts. (iii) The task was intentionally low-stakes and time-bounded, which may influence subjective ratings and interaction strategies compared to real production settings. Future research should therefore examine ecological validity through higher-stakes, longer-horizon deployments (e.g., course projects or studio-like briefs), and longitudinal studies to assess how RPGAgent affects skill development, workflow adoption, and designers' evolving sense of agency over time.

7 Conclusion

In this work, we introduce RPGAgent, a collaborative multi-agent framework that integrates large language models and PCG techniques for the holistic creation of RPG game content. By orchestrating specialized agents responsible for narrative, scene design, game mechanics, and code generation, RPGAgent not only achieves efficient division of labor, but also enables dynamic coordination and feedback among agents. Central to this collaboration is the use of structured outputs as standardized interfaces between agents; this design ensures that the diverse components they generate are seamlessly interconnected, thereby maintaining semantic coherence and thematic continuity throughout the design pipeline. In this way, the framework both supports creative diversity at each stage and guarantees consistency across the entire content generation process.

RPGAgent thus highlights the profound potential of multi-agent collaboration in game development. By coupling the unique strengths of intelligent agents with the generative flexibility of PCG, and ensuring tight integration through structured outputs-the framework demonstrates how complex and interdependent game elements can be produced in a controlled yet creative manner. This approach inspires a new paradigm for automated game design, in which coordinated multi-agent systems, grounded in structured communication, foster both scalability and coherence, offering promising

avenues for the future evolution of co-creative digital content generation.

Acknowledgments

This work is supported by the Lushan Lab Research Funding (Z202567450382), the Science and Technology Innovation Program of Hunan Province (Grant No. 2025RC6002), and the Hunan Provincial Natural Science Foundation (Grant No. 2025JJ50400).

References

[1] Seyed Hossein Alavi, Weijia Xu, Nebojsa Jojic, Daniel Kennett, Raymond T. Ng, Sudha Rao, Haiyan Zhang, Bill Dolan, and Vered Shwartz. 2024. Game Plot Design with an LLM-powered Assistant: An Empirical Study with Game Designers. arXiv:2411.02714 [cs.CL] https://arxiv.org/abs/2411.02714

[2] Saeed Amiri-Chimeh, Hassan Haghighi, and Mojtaba Vahidi-Asl. 2024. JAHAN: A framework for procedural generation of game maps from design specifications. *Entertainment Computing* 50 (2024), 100644.

[3] Asad Anjum, Yuting Li, Noelle Law, Megan Charity, and Julian Togelius. 2024. The ink splotch effect: A case study on chatgpt as a co-creative game designer. In *Proceedings of the 19th International Conference on the Foundations of Digital Games*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3649921.3650010

[4] J Timothy Balint and Rafael Bidarra. 2022. Procedural generation of narrative worlds. *IEEE Transactions on Games* 15, 2 (2022), 262–272.

[5] Jim Bizzocchi and Joshua Tanenbaum. 2011. Well read: Applying close reading techniques to gameplay experiences. *Well played* 3, 3 (2011), 289–315.

[6] Jim Bizzochi. 2007. Games and narrative: An analytical framework. *Loading...* 1, 1 (2007).

[7] Philip Bontrager, Aditi Roy, Julian Togelius, Nasir Memon, and Arun Ross. 2018. Deepmasterprints: Generating masterprints for dictionary attacks via latent variable evolution. In *2018 IEEE 9th International Conference on Biometrics Theory, Applications and Systems (BTAS)*. IEEE, 1–9.

[8] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.

[9] Virginia Braun and Victoria Clarke. 2021. One size fits all? What counts as quality practice in (reflexive) thematic analysis? *Qualitative research in psychology* 18, 3 (2021), 328–352.

[10] Dake Chen, Haoyang Zhang, Hanbin Wang, Yunhao Huo, Yuzhao Li, and Junjie Wang. 2025. GameGPT: Multi-agent Collaborative Framework for Game Development. arXiv:2310.08067 [cs.AI] https://arxiv.org/abs/2310.08067

[11] Erin Cherry and Celine Latulipe. 2014. Quantifying the creativity support of digital tools through the creativity support index. *ACM Transactions on Computer-Human Interaction (TOCHI)* 21, 4 (2014), 1–25.

[12] Steve Dahlskog and Julian Togelius. 2012. Patterns and procedural content generation: Revisiting mario in world 1 level 1. In *Proceedings of the First Workshop on Design Patterns in Games*. 1–8.

[13] Steve Dahlskog, Julian Togelius, and Mark J Nelson. 2014. Linear levels through n-grams. In *Proceedings of the 18th International Academic MindTrek Conference: Media Business, Management, Content & Services*. 200–206.

[14] Sam Earle, Samyak Parajuli, and Andrzej Banburski-Fahey. 2025. DreamGarden: A Designer Assistant for Growing Games from a Single Prompt. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–19.

[15] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. 2024. Large language models and games: A survey and roadmap. *IEEE Transactions on Games* (2024).

[16] Matthew Guzdial, Brent Harrison, Boyang Li, and Mark O Riedl. 2015. Crowdsourcing Open Interactive Narrative.. In *Foundations of Digital Games*.

[17] Matthew Guzdial, Nicholas Liao, Jonathan Chen, Shao-Yu Chen, Shukan Shah, Vishwa Shah, Joshua Reno, Gillian Smith, and Mark O Riedl. 2019. Friend, collaborator, student, manager: How design of an ai-driven game level editor affects creators. In *Proceedings of the 2019 CHI conference on human factors in computing systems*. 1–13.

[18] Ken Hartsook, Alexander Zook, Sauvik Das, and Mark O Riedl. 2011. Toward supporting stories with procedurally generated game worlds. In *2011 IEEE Conference on Computational Intelligence and Games (CIG'11)*. IEEE, 297–304.

[19] Mark Hendrikx, Sebastiaan Meijer, Joeri Van Der Velden, and Alexandru Iosup. 2013. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)* 9, 1 (2013), 1–22.

[20] Jiale Hong, Hongqiu Wu, and Hai Zhao. 2024. Game Development as Human-LLM Interaction. *arXiv preprint arXiv:2408.09386* (2024).

[21] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VtmBAGCN7o

[22] Chengpeng Hu, Yunlong Zhao, and Jialin Liu. 2024. Game generation via large language models. In *2024 IEEE Conference on Games (CoG)*. IEEE, 1–4.

[23] Panwen Hu, Jin Jiang, Jianqi Chen, Mingfei Han, Shengcai Liao, Xiaojun Chang, and Xiaodan Liang. 2024. StoryAgent: Customized Storytelling Video Generation via Multi-Agent Collaboration. arXiv:2411.04925 [cs.CV] https://arxiv.org/abs/2411.04925

[24] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2024. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 [cs.CL] https://arxiv.org/abs/2312.13010

[25] Robin Hunicke, Marc LeBlanc, Robert Zubek, et al. 2004. MDA: A formal approach to game design and game research. In *Proceedings of the AAAI Workshop on Challenges in Game AI*, Vol. 4. San Jose, CA, 1722.

[26] Fantine Huot, Reinald Kim Amplayo, Jennimaria Palomaki, Alice Shoshana Jakobovits, Elizabeth Clark, and Mirella Lapata. 2025. Agents' Room: Narrative Generation through Multi-step Collaboration. arXiv:2410.02603 [cs.CL] https://arxiv.org/abs/2410.02603

[27] Nanna Inie, Jeanette Falk, and Steve Tanimoto. 2023. Designing participatory ai: Creative professionalsâĂŹ worries and expectations about generative ai. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–8.

[28] Andrés Isaza-Giraldo, Paulo Bala, Pedro F Campos, and Lucas Pereira. 2024. Prompt-gaming: A pilot study on llm-evaluating agent in a meaningful energy game. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–12.

[29] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. arXiv:2405.11403 [cs.CL] https://arxiv.org/abs/2405.11403

[30] Ray Ito and Junichiro Takahashi. 2024. Game Agent Driven by Free-Form Text Command: Using LLM-based Code Generation and Behavior Branch. *arXiv preprint arXiv:2402.07442* (2024).

[31] Nicholas Jennings, Han Wang, Isabel Li, James Smith, and Bjoern Hartmann. 2024. What's the Game, then? Opportunities and Challenges for Runtime Behavior Generation. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. 1–13.

[32] Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L'elio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *ArXiv* abs/2310.06825 (2023). https://api.semanticscholar.org/CorpusID:263830494

[33] Jesper Juul. 2005. Games telling stories. *Handbook of computer game studies* (2005), 219–226.

[34] Anssi Kanervisto, Dave Bignell, Linda Yilin Wen, Martin Grayson, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Tabish Rashid, Tim Pearce, Yuhan Cao, et al. 2025. World and human action models towards gameplay ideation. *Nature* 638, 8051 (2025), 656–663.

[35] Jan Kruse, Andy M Connor, and Stefan Marks. 2021. An interactive multi-agent system for game design. *The Computer Games Journal* 10, 1 (2021), 41–63.

[36] Bettina Laugwitz, Theo Held, and Martin Schrepp. 2008. Construction and evaluation of a user experience questionnaire. In *Symposium of the Austrian HCI and usability engineering group*. Springer, 63–76.

[37] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems* 36 (2023), 51991–52008.

[38] Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. 2024. Auto mc-reward: Automated dense reward design with large language models for minecraft. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16426–16435.

[39] Joakim Lindfors. 2025. Leveraging Generative AI to Create Themed Assets for Games: A Study on Narrative and 3D Asset Creation With AI.

[40] Craig A Lindley. 2005. Story and narrative structures in computer games. *Bushoff, Brunhild. ed* (2005).

[41] Daniel Mackay. 2017. *The fantasy role-playing game: A new performing art*. McFarland.

[42] Diego Maldonado, Edison Cruz, Jackeline Abad Torres, Patricio J Cruz, and Silvana del Pilar Gamboa Benitez. 2024. Multi-agent systems: A survey about its components, framework and workflow. *IEEE Access* 12 (2024), 80950–80975.

[43] Mahdi Farrokhi Maleki and Richard Zhao. 2024. Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 20. 167–178.

[44] Xinyu Mao, Wanli Yu, Kazunori D Yamada, and Michael R. Zielewski. 2024. Procedural Content Generation via Generative Artificial Intelligence.

arXiv:2407.09013 [cs.AI] https://arxiv.org/abs/2407.09013

[45] Nicola Marsden, Annelie Rothe-Wulf, Yvonne Tang, Claudia Herling, and Tim Reichert. 2025. Navigating Bias: Using LLMs to Analyze Discrimination in Entrepreneurial Game Dialogues. In *International Conference on Human-Computer Interaction*. Springer, 202–219.

[46] Aditi Mishra, Frederik Brudy, Qian Zhou, George Fitzmaurice, and Fraser Anderson. 2025. WhatIF: Branched Narrative Fiction Visualization for Authoring Emergent Narratives using Large Language Models. In *Proceedings of the 2025 Conference on Creativity and Cognition*. 590–605.

[47] Sajad Mohaghegh, Mohammad Amin Ramezan Dehnavi, Golnoosh Abdollahinejad, and Matin Hashemi. 2023. PCGPT: Procedural Content Generation via Transformers. arXiv:2310.02405 [cs.LG] https://arxiv.org/abs/2310.02405

[48] Muhammad U. Nasir, Steven James, and Julian Togelius. 2024. Word2World: Generating Stories and Worlds through Large Language Models. doi:10.48550/arXiv.2405.06686 arXiv:2405.06686 [cs] TLDR: This work introduces Word2World, a system that enables LLMs to procedurally design playable games through stories, without any task-specific fine-tuning..

[49] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[50] Xiangyu Peng, Jessica Quaye, Sudha Rao, Weijia Xu, Portia Botchway, Chris Brockett, Nebojsa Jojic, Gabriel DesGarennes, Ken Lobb, Michael Xu, et al. 2024. Player-driven emergence in llm-driven game narrative. In *2024 IEEE Conference on Games (CoG)*. IEEE, 1–8.

[51] Leonardo Tortoro Pereira, Paulo Victor de Souza Prado, Rafael Miranda Lopes, and Claudio Fabiano Motta Toledo. 2021. Procedural generation of dungeonsâĂŹ maps and locked-door missions through an evolutionary algorithm validated with players. *Expert Systems with Applications* 180 (2021), 115009.

[52] Leonardo T Pereira, Breno MF Viana, and Claudio FM Toledo. 2021. Procedural enemy generation through parallel evolutionary algorithm. In *2021 20th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*. IEEE, 126–135.

[53] Adhistya Erna Permanasari, Bernadeta Ratna PS, Mirza Putri Maharani, Sunu Wibirama, Junaedy Yunus, et al. 2021. Design of Gamification for Anatomy Learning Media. In *2021 13th International Conference on Information Technology and Electrical Engineering (ICITEE)*. IEEE, 122–126.

[54] Pankaj Pilaniwala, Girish Chhabra, and Prabhleen Kaur. 2024. The future of game development in the era of gen ai. In *2024 Artificial Intelligence for Business (AIxB)*. IEEE, 39–42.

[55] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15174–15186.

[56] Anurag Sarkar, Adam Summerville, Sam Snodgrass, Gerard Bentley, and Joseph Osborn. 2020. Exploring level blending across platformers via paths and affordances. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 16. 280–286.

[57] Jesse Schell. 2008. *The Art of Game Design: A book of lenses*. CRC press.

[58] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM Agents as Research Assistants. arXiv:2501.04227 [cs.HC] https://arxiv.org/abs/2501.04227

[59] Noor Shaker, Julian Togelius, and Mark J Nelson. 2016. Procedural content generation in games. (2016).

[60] Ben Shneiderman. 2007. Creativity support tools: accelerating discovery and innovation. *Commun. ACM* 50, 12 (2007), 20–32.

[61] Tianye Shu, Jialin Liu, and Georgios N Yannakakis. 2021. Experience-driven PCG via reinforcement learning: A Super Mario Bros study. In *2021 IEEE Conference on Games (CoG)*. IEEE, 1–9.

[62] Anthony J Smith and Joanna J Bryson. 2014. A logical approach to building dungeons: Answer set programming for hierarchical procedural content generation in roguelike games. In *Proceedings of the 50th Anniversary Convention of the AISB*. 16.

[63] Sam Snodgrass and Santiago Ontanón. 2016. Controllable Procedural Content Generation via Constrained Multi-Dimensional Markov Chain Sampling.. In *IJCAI*. 780–786.

[64] SongTang SongTang, Kaiyong Zhao, Lei Wang, Yuliang Li, Xuebo Liu, Junyi Zou, Qiang Wang, and Xiaowen Chu. 2025. UnrealLLM: Towards Highly Controllable and Interactable 3D Scene Generation by LLM-powered Procedural Content Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 19417–19435.

[65] Sasha Soraine and Jacques Carette. 2025. The Many Views of Game-Related Experiences with the Experiential Tetrad. In *Proceedings of the 20th International Conference on the Foundations of Digital Games*. 1–11.

[66] Adam Summerville and Michael Mateas. 2016. Mystical tutor: A magic: The gathering design assistant via denoising sequence-to-sequence learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 12. 86–92.

[67] Adam Summerville and Michael Mateas. 2016. Super Mario as a String: Platformer Level Generation Via LSTMs. arXiv:1603.00930 [cs.NE] https://arxiv.org/abs/1603.00930

[68] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. 2018. Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games* 10, 3 (2018), 257–270.

[69] Julian Togelius, Emil Kastbjerg, David Schedl, and Georgios N Yannakakis. 2011. What is procedural content generation? Mario on the borderline. In *Proceedings of the 2nd international workshop on procedural content generation in games*. 1–6.

[70] Julian Togelius, Georgios N Yannakakis, Kenneth O Stanley, and Cameron Browne. 2011. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games* 3, 3 (2011), 172–186.

[71] Ruben Rodriguez Torrado, Ahmed Khalifa, Michael Cerny Green, Niels Justesen, Sebastian Risi, and Julian Togelius. 2020. Bootstrapping conditional gans for video game level generation. In *2020 IEEE Conference on Games (CoG)*. IEEE, 41–48.

[72] César Villacís, Walter Fuertes, Mónica Santillán, Hernán Aules, Ana Tacuri, Margarita Zambrano, and Edgar Salguero. 2016. On the development of strategic games based on a semiotic analysis: a case study of an optimized tic-tac-toe. In *International Conference on Enterprise Information Systems*, Vol. 2. SCITEPRESS, 425–432.

[73] Vanessa Volz, Jacob Schrum, Jialin Liu, Simon M Lucas, Adam Smith, and Sebastian Risi. 2018. Evolving mario levels in the latent space of a deep convolutional generative adversarial network. In *Proceedings of the genetic and evolutionary computation conference*. 221–228.

[74] Samangi Wadinambiarachchi, Ryan M Kelly, Saumya Pareek, Qiushi Zhou, and Eduardo Velloso. 2024. The effects of generative ai on design fixation and divergent thinking. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–18.

[75] Wolfgang Walk, Daniel Görlich, and Mark Barrett. 2017. Design, dynamics, experience (DDE): an advancement of the MDA framework for game design. In *Game dynamics: Best practices in procedural and dynamic game content generation*. Springer, 27–45.

[76] Chaoguang Wang and Lusha Huang. 2021. A Systematic Review of Serious Games for Collaborative Learning: Theoretical Framework, Game Mechanic and Efficiency Assessment. *International Journal of Emerging Technologies in Learning* 16, 6 (2021).

[77] Xiyuan Wang, Yi-Fan Cao, Junjie Xiong, Sizhe Chen, Wenxuan Li, Junjie Zhang, and Quan Li. 2025. ClueCart: Supporting Game Story Interpretation and Narrative Inference from Fragmented Clues. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–26.

[78] Yi Wang, Qian Zhou, and David Ledo. 2024. StoryVerse: Towards co-authoring dynamic plot with LLM-based character simulation via narrative planning. In *Proceedings of the 19th International Conference on the Foundations of Digital Games*. 1–4.

[79] Kaijie Xu and Clark Verbrugge. 2025. Constraint Is All You Need: Optimization-Based 3D Level Generation with LLMs. In *Proceedings of the 20th International Conference on the Foundations of Digital Games*. 1–13.

[80] Xuenan Xu, Jiahao Mei, Chenliang Li, Yuning Wu, Ming Yan, Shaopeng Lai, Ji Zhang, and Mengyue Wu. 2025. MM-StoryAgent: Immersive Narrated Storybook Video Generation with a Multi-Agent Paradigm across Text, Image and Audio. arXiv:2503.05242 [cs.CL] https://arxiv.org/abs/2503.05242

[81] Zhenran Xu, Jifang Wang, Longyue Wang, Zhouyi Li, Senbao Shi, Baotian Hu, and Min Zhang. 2024. Filmagent: Automating virtual film production through a multi-agent collaborative framework. In *SIGGRAPH Asia 2024 Technical Communications*. 1–4.

[82] Daijin Yang, Erica Kleinman, and Casper Harteveld. 2024. GPT for Games: A Scoping Review (2020-2023). In *2024 IEEE Conference on Games (CoG)*. IEEE, 1âĂŞ8. doi:10.1109/cog60054.2024.10645548

[83] Chunyang Yu, Pai Zheng, Tao Peng, Xun Xu, Steven Vos, and Xipei Ren. 2025. Design meets AI: challenges and opportunities. *Journal of Engineering Design* 36, 5-6 (2025), 637–641. doi:10.1080/09544828.2025.2484085

[84] José P Zagal and Sebastian Deterding. 2018. Definitions of âĂIJrole-playing gamesâĂİ. In *Role-playing game studies*. Routledge, 19–51.

[85] Marvin Zammit, Antonios Liapis, and Georgios N Yannakakis. 2024. CrawLLM: theming games with large language models. In *2024 IEEE Conference on Games (CoG)*. IEEE, 1–2.

[86] Mengqi Zhou, Yuxi Wang, Jun Hou, Chuanchen Luo, Zhaoxiang Zhang, and Junran Peng. 2024. SceneX:Procedural Controllable Large-scale Scene Generation via Large-language Models. http://arxiv.org/abs/2403.15698 arXiv:2403.15698 [cs].

A LLM Prompts for Multi-Agent Workflow in RPGAgent

A.1 Prompts of Narrative Designer Agent

```
You are the Narrative Agent. Your task is to expand a user'
     s RPG story idea into a complete background narrative
     and divide it into sequential story steps.

Input:
- story_outline (string): A short description of the RPG
     theme or premise.
- design_notes (string): The user's preferences and any
     special settings or constraints.

Rules:
- Step 1 must include the MainCharacter.
- Steps 2+ exclude the MainCharacter; focus on enemies,
     NPCs, or items.
- Each step must have a unique combination of characters.
- Character types: Enemy, NPC (friendly), Items.
- step.title should be concise, engaging, in English.
- location (int):
  0=Village, 1=Forest, 2=Grassland, 3=Castle, 4=Cave
- Locations follow logical progression and cover varied
     environments.
- Output in JSON with valid syntax.

Output example:
{
  "story": "A young hero sets out from the village to save
     the kingdom from an ancient evil...",
  "steps": [
    {
      "step": 1,
      "title": "The Beginning: New Adventure",
      "location": 0,
      "objective": "Meet the village elder to receive your
     quest.",
      "keyCharacters": ["Hero", "Village Elder"],
      "mainDialogues": [
        { "character": "Village Elder", "dialogue": "
     Welcome, traveler. The kingdom needs you." }
      ],
      "keyItems": ["Map", "Sword"]
    }
  ]
}
```

A.2 Prompts of Scene Designer Agent

```
You are the Scene Agent. Your task is to generate Unity map
      parameters and allocate game assets for each
     narrative step.

Input:
{
  "story": "Optional background setting and theme.",
  "steps": [
    { "step": 1, "location": 0, "objective": "...", "
     keyCharacters": [...], "keyItems": [...] }
  ],
  "assets": [
    { "name": "Village Elder", "type": "NPC", "description
     ": "Old wise man..." }
  ]
}

Rules:
- Output scene_params: noiseScale, landThreshold,
     mountainThreshold with numeric values.
- Asset allocation per step:
  Step 1: may include 1 MainCharacter + NPC/Enemy + Items
  Steps 2+: no MainCharacter, >= 1 items
  >= 2 assets each step
- Asset matching priority: name > alias > description >
     best semantic/type
- All output asset names must match exactly existing assets
     .

Output example:
{
  "sceneParams": { "noiseScale": 20, "landThreshold": 0.15,
      "mountainThreshold": 0.80 },
  "selectedAssets": [
    { "step": 1, "assets": ["MainHero", "Village Elder", "
     Map"] },
    { "step": 2, "assets": ["Forest Guardian", "Ancient
     Tree"] }
  ]
}
```

A.3 Prompts of Mechanics Designer Agent

```
You are the Mechanics Agent. Your task is to design RPG
     gameplay mechanics for a list of game assets.

Input:
[
  { "name": "Village Elder", "type": "NPC", "aliases": [],
     "description": "Old wise man..." }
]

Rules:
- Only design mechanics for the provided input assets.
- name and type must exactly match input.
- Output JSON array where each element contains a "
     mechanics" object:
  - function: Role/purpose in the game
  - interactionMode: Player interaction method
  - outcome: Result of the interaction
  - systemRelationships: Related game systems

Output example:
[
  {
    "type": "NPC",
    "name": "Village Elder",
    "mechanics": {
      "function": "Provides quests and lore hints to
     players",
      "interactionMode": "Dialogue when player is in range
     ",
      "outcome": "Player receives quest item and objectives
     ",
      "systemRelationships": "Linked to quest and dialogue
     systems"
    }
  }
]
```

A.4 Prompts of Code Generator Agent

```
You are the CodeGen Agent. Your task is to assign gameplay
     mechanics to scene objects and adapt Unity C#
     templates based on the given mechanics design.

Input:
{
  "sceneObjects": [
    { "name": "Village Elder", "type": "NPC", "displayName
     ": "Village Elder", "description": "Old wise man..." }
  ],
  "requirements": {
    "function": "Greets the player, provides hints, and
     offers a quest.",
    "interactionMode": "Dialogues triggered by E key within
      range.",
    "outcome": "Player receives quest objectives and quest
     item.",
    "systemRelationships": "Quest and dialogue systems."
  },
  "codeTemplate": {
    "Dialogue": "using UnityEngine; // ... base NPC
     dialogue template ..."
  }
}

Rules:
- Output must be one valid JSON object, containing:
  - objectMechanics: mapping asset name to mechanic type.
  - dialogueLines / params: customized from requirements.
  - modifiedCode: adapted from the provided template.

Output example:
{
  "objectMechanics": { "Village Elder": "Dialogue" },
  "dialogueLines": {
    "Village Elder": [
      "Welcome, young traveler.",
      "The forest ahead hides danger and reward.",
      "Seek the shrine beyond the hills."
    ]
  },
  "modifiedCode": {
    "Dialogue": "using UnityEngine; // Modified from
     template to include E-key trigger within range."
  }
}
```